AI Agent Rules

This document provides specific rules for AI agents (like Cursor/Console AI) when working on this repository.

CRITICAL: Before Running Any Python or Test Command: Read the "Python Script Execution Rules" and "Syntax Validation" sections in this document and use the specified paths. Do NOT assume `python` or `py` will work. See also `.cursor/rules/` for mandatory project rules (Python paths, debugging).

Discussion Mode Rules

CRITICAL: "Discuss" or "Discuss Only" Means No File Updates

When user says "Discuss" or "Discuss only":

- **DO NOT** update any files (code, markdown, configuration, etc.)
- **DO NOT** create new files
- **DO NOT** modify existing files
- **Exception**: Discussion markdown files in `DevNotes/Discussions/` may be updated if explicitly part of the discussion

What "Discuss" Means:

- Provide analysis, recommendations, and discussion
- Explain concepts, approaches, and trade-offs
- Answer questions and clarify understanding
- Propose solutions and alternatives
- **DO NOT** implement changes

When to Update Files:

- Only after explicit user instruction to proceed with implementation
- User will request file updates as a separate step
- Implementation happens after discussion and agreement

Agent Behavior:

1. **During Discussion**: Provide analysis, recommendations, and discussion only
2. **After Discussion**: Wait for explicit user instruction to proceed with implementation
3. **File Updates**: Only update files when explicitly requested, not during discussion phase

Rationale:

- Discussion phase is for understanding and agreement
- Implementation phase is separate and explicit

- Prevents premature changes before full understanding
- Allows user to review and approve approach before changes

Project Architecture Overview

CRITICAL: This project is web-first. CLI interface has been deprecated and removed.

- **Primary Interface:** Web dashboard (Django application)
- **No CLI Interface:** `main_cli.py` and CLI-related code have been removed
- **Core Application:** `contest_tools/` - Independent, reusable library (no CLI dependencies)
- **Web Application:** `web_app/` - Django application using core library directly
- **Test Infrastructure:** `test_code/` - Separate from main project, uses Django test client

Agent Behavior:

- Do NOT create or reference CLI interfaces
- Do NOT assume CLI functionality exists
- Use web interface or Django test client for testing
- Core application (`contest_tools/`) is independent and has no CLI dependencies

---

Git Operations Overview

Primary Principle: AI assists with git operations, but the user maintains final control over critical operations.

What AI Can Do Directly

[OK] Commit Message Generation

- Generate Angular-formatted commit messages following strict format
- Suggest appropriate commit types (feat, fix, docs, etc.)
- Format commit messages with proper structure (subject, blank line, body)
- Use multiple `-m` flags or single `-m` with escaped newlines for proper formatting

[OK] Feature Branch Commits

- Create commits on feature branches

- Use informal messages during development ("wip", "refactor X")
- Commit code changes directly to feature branches

[OK] Commit Workflow - Check for Uncommitted Files

AI agents MUST check for uncommitted changes before performing a commit.

Rule: When asked to commit changes, AI agents must:

1. **Check git status** for untracked or modified files in the working directory
2. **Identify** which files are part of the current change vs. unrelated modifications
3. **Ask the user** if there are untracked/modified files that aren't being included in the commit
4. **Offer options** for handling untracked files (commit, delete, add to .gitignore, or leave untracked)
5. **Wait for user confirmation** before proceeding with partial commits

Typical Behavior:

- **Default:** Include all project files in a commit (whether part of current change or previous work)
- **Exception:** Only exclude files if explicitly requested by user (e.g., "commit only X, ignore Y")

Agent Behavior:

- Before committing, run `git status` or `git status --short` to check for uncommitted changes
- If uncommitted files exist beyond what's being committed, present options:
  - "There are uncommitted files (X, Y, Z) that aren't being included in this commit. Would you like to:
    1. Include them in this commit
    2. Commit them separately
    3. Delete them
    4. Add them to .gitignore (if they should be ignored)
    5. Leave them as untracked/unstaged"
- If untracked files exist and aren't being committed, **offer to add them to .gitignore** if appropriate
- Only proceed with partial commits if user explicitly requests it

Handling Untracked Files:

- **If files should be committed:** Add them to the commit (either in current commit or separate commit)
- **If files should be ignored:** Offer to add patterns to `.gitignore` (e.g., `test_code/`, `*.log`, etc.)
- **If files should be deleted:** Ask for confirmation before deleting

- **If files should remain untracked:** Acknowledge and proceed with commit

Rationale:

- Prevents leaving unrelated changes uncommitted
- Ensures all project files are tracked and committed appropriately
- Helps maintain `.gitignore` to exclude files that shouldn't be tracked
- Maintains clean working tree after commits

[OK] Generate Commands

- Suggest git commands for workflows
- Generate merge commands with `--no-ff` flag
- Create command sequences for complex operations

[OK] Automatic Operations

- Pre-commit hooks automatically update `version.py` git hash
- No manual intervention needed for hash updates

What Requires User Review/Approval

WARNING: Merges to Main Branch

Process:

1. AI generates merge command: `git merge --no-ff feat/branch-name`
2. AI generates merge commit message in Angular format
3. **User reviews** merge message in editor
4. **User saves/closes** editor to complete merge
5. **User pushes** when ready

Why: Main branch is sacred - requires human oversight

WARNING: Creating Tags/Releases

Process:

1. AI suggests version number based on commit types
2. AI suggests update to `version.py`
3. **User confirms** version number
4. AI generates commands:
   ```
   # Update version.py __version__ to match tag
   # Commit version change
   git tag v1.0.0-beta.5
   git push origin v1.0.0-beta.5
   ```
5. **User reviews** and executes commands

6. **User confirms** tag creation

Why: Releases are critical - require explicit approval

WARNING: Pushing to Remote

Process:

1. AI suggests push commands
2. **User executes** push commands manually
3. AI should NOT auto-push without explicit user request

Why: Security control - prevents accidental remote changes

WARNING: Hotfix Operations

Process:

1. AI generates hotfix workflow commands
2. **User reviews** each step
3. **User executes** step by step
4. User confirms each critical operation (tag, push)

Why: Hotfixes are high-risk - require careful manual review

Character Encoding Rules

7-bit ASCII Requirement

AI agents MUST use 7-bit ASCII (characters 0-127 only) in all non-markdown files.

MANDATORY: Check this section BEFORE creating or modifying any code files, scripts, or configuration files.

MANDATORY WORKFLOW FOR FILE MODIFICATIONS:

1. **BEFORE modifying any file:**
   - Check this section of AI Agent Rules
   - Identify if the file type requires 7-bit ASCII (all non-markdown files)
   - **Scan the file for existing Unicode/emoji violations** (use grep/search tools if needed)
   - Plan replacements for any non-ASCII characters using the common replacements list
   - **Fix existing violations BEFORE making other changes** to the file
2. **DURING modification:**
   - **Always** use 7-bit ASCII characters (0-127 only)

- **Never** add emoji, Unicode symbols, or multi-byte characters
- **Replace** any existing non-ASCII characters with ASCII equivalents
- **Use** the common replacements listed below

3. **AFTER modification:**
   - Verify the file contains only 7-bit ASCII characters
   - Check for any Unicode/emoji that may have been introduced
   - If violations found: Fix immediately before proceeding

Critical Rule: When creating or modifying any code files, scripts, or configuration files, AI agents must:

- [OK] Use only 7-bit ASCII characters
- [OK] Replace any non-ASCII characters with ASCII equivalents

☒ Never use emoji, Unicode symbols, or multi-byte characters in code/scripts

Files Affected:

- All source code files (`.py`, `.js`, `.ts`, `.html`, `.css`, etc.)
- All script files (`.ps1`, `.bat`, `.sh`, etc.)
- All configuration files (`.json`, `.yaml`, `.ini`, etc.)
- Documentation source files (`.rst`, `.txt`, etc.)

Exceptions:

- **Markdown files (`.md`):** May contain UTF-8 characters, BUT with PDF compatibility restrictions (see below).
- **Third-party/vendor libraries:** Minified or vendor-supplied files (e.g., `html2canvas.min.js`) are excluded from this rule. Only project-authored code must comply.

Scanning for Violations: Before modifying files, use search tools to identify existing violations:

- Search for common Unicode characters: `grep -r "[ •→ ]" path/to/directory`
- Check specific file types: `grep -r "pattern" --include="*.py" path/`
- Focus on files you're about to modify: Always scan target files before editing

Runtime Error Risk: Non-ASCII characters in code can cause runtime errors:

- `UnicodeEncodeError` when printing emoji in Windows console (e.g., `'charmap' codec can't encode characters`)
- Parsing errors in PowerShell/batch scripts
- Import failures when Python source files contain non-ASCII characters
- **Always fix violations immediately** to prevent runtime failures

PDF Compatibility Rule for Markdown:

- **All markdown files MUST be PDF-compatible** (no emoji or Unicode symbols that break LaTeX/PDF conversion)
- **Use plain text alternatives** instead of emoji:
  - [OK] (checkmark) → `[OK]`, `OK`, or `PASS`
  - ☒ (cross mark) → `[X]`, `FAIL`, or `ERROR`
  - WARNING: (warning) → `WARNING:` or `WARN:`
  - ∗ (bullet) → `*` or `-`
  - Major Features: (target) → `Major Features:` or `Features:`
  - Enhancements: (sparkles) → `Enhancements:` or `Improvements:`
  - Bug Fixes: (bug) → `Bug Fixes:`
  - Documentation: (books) → `Documentation:`
  - Technical: (wrench) → `Technical:` or `Technical Improvements:`
  - Statistics: (chart) → `Statistics:` or `Metrics:`
  - Migration: (arrows) → `Migration:` or `Migration Notes:`
  - Deployment: (rocket) → `Deployment:` or `Deployment Notes:`
  - Notes: (memo) → `Notes:` or `Commit History:`
- **Rationale:** LaTeX fonts (e.g., lmroman10-regular) used for PDF conversion do not support emoji characters, causing warnings and formatting issues.

Common Replacements:

- (checkmark) → `[OK]`, `OK`, or `PASS`
- (cross mark) → `[X]`, `FAIL`, or `ERROR`
- (warning) → `WARNING:` or `WARN:`
- • (bullet) → `*` or `-`
- → (arrow) → `->` or `=>` or `:`
- (stopwatch) → `[TIME]` or `[PROFILE]` or remove entirely

Rationale: PowerShell, batch files, and many other tools can fail to parse files correctly when multi-byte UTF-8 characters are present, leading to cryptic parsing errors. Additionally, PDF conversion tools require plain text alternatives to emoji.

Commit Message Standards

Standard: All commits MUST follow strict Angular Conventional Commits format, regardless of branch.

Angular Format Structure (Mandatory)

```
<type>(<scope>): <short summary>

<body>

<footer> (optional, only for BREAKING CHANGE)
```

Requirements:

- **Type:** One of: feat, fix, docs, style, refactor, perf, test, build, ci, chore
- **Scope:** Optional, lowercase, in parentheses (e.g., `(readme)`, `(dashboard)`)
- **Subject:** Imperative, lowercase, < 50 characters
- **Blank line:** MANDATORY between subject and body
- **Body:** Detailed explanation, wrap at 72 characters, bullet points allowed
- **Footer:** Only for BREAKING CHANGE or issue tracking

For Feature Branch Commits

- **Must follow Angular format strictly**
- Use appropriate type (feat, fix, docs, refactor, etc.)
- Include body with bullet points describing changes
- Example:

```
feat(dashboard): add user dashboard with widgets

Implements main dashboard view with widget framework.
Adds real-time data updates and user preferences.

- Created dashboard view component
- Implemented widget system
- Added API integration for live data
```

For Merge Commits to Main

- **Must follow Angular format strictly:**

```
feat(dashboard): add user dashboard with widgets

Implements main dashboard view with widget framework.
Adds real-time data updates and user preferences.

- Created dashboard view component
- Implemented widget system
- Added API integration for live data
```

- **Subject:** User-facing feature description
- **Body:** What the feature does (not implementation details)
- **Format:** Lowercase, imperative, < 50 chars for subject

For Direct Commits to Main (Rare)

- Follow Angular format strictly
- Use appropriate type (fix, docs, chore)
- Include detailed body for significant changes

Version Management

CRITICAL: Document Consistency Check

MANDATORY: Before performing any version updates, AI agents MUST:

1. **Read both documents:**
   - `Docs/AI_AGENT_RULES.md` Section "Version Management" (this section)
   - `Docs/VersionManagement.md` Section 3 (Documentation Versioning Policy)
2. **Verify consistency using this checklist:**
   - ☐ Category definitions match (A, B, C, D)
   - ☐ Category file lists match
   - ☐ Workflow steps align (same steps, same order)
   - ☐ Policy rules are consistent (when to update, how to update)
   - ☐ "Compatible With" field rules match
   - ☐ Revision history requirements match
   - ☐ File locations match
   - ☐ Version number formats match
   - ☐ Update triggers match (when to update each category)
3. **If ANY discrepancy is detected:**
   - **STOP immediately**
   - **DO NOT proceed with version updates**
   - **Report the discrepancy using this format:**

```
**DISCREPANCY DETECTED - STOPPING WORKFLOW**

I detected a discrepancy between the versioning documents:

**Location:** [Section name in each document]
**Issue:** [Brief description]
**AI_AGENT_RULES.md says:** [Exact quote or summary]
**VersionManagement.md says:** [Exact quote or summary]

**Impact:** [What could go wrong if I proceed]

**Request for Guidance:**
1. Which document should I follow?
2. Should I update one document to match the other?
3. Is this an intentional difference, or a documentation error?

**Action:** Waiting for your guidance before proceeding with version updates.
```

   - **Wait for explicit user guidance**

For complete discrepancy detection procedures, see: `Docs/VersionManagement.md` Section 5

How Version.py Works

- `contest_tools/version.py` contains `__version__` and `__git_hash__`
- Pre-commit hook automatically updates `__git_hash__` before each commit
- `__version__` should match most recent git tag (manual update before releases)

Release Workflow: Creating a New Version Tag

Purpose: Ensure version consistency across `version.py`, documentation, and git tags when creating a new release.

When to Use: Before creating any version tag (alpha, beta, or release).

Step-by-Step Process 0. Document Consistency Check (MANDATORY FIRST STEP)

CRITICAL: This step MUST be performed before any version updates.

a. **Read both versioning documents:**

- `Docs/AI_AGENT_RULES.md` Section "Version Management" (this section)
- `Docs/VersionManagement.md` Section 3 (Documentation Versioning Policy)

b. **Verify consistency:**

- Use the discrepancy detection checklist above
- Check that category definitions, file lists, workflow steps, and policy rules match

c. **If discrepancy detected:**

- STOP immediately
- Report using the format above
- Wait for user guidance

d. **Only proceed if documents are consistent**

1. Pre-Release Checklist

☐ Verify current branch state (`git status`)
☐ Ensure all feature work is committed on feature branch (code, tests, etc.)
☐ Review recent commits to determine appropriate version increment
☐ Confirm target version number with user (e.g., `v1.0.0-alpha.3`)

2. Create Draft Release Notes, Announcement, and CHANGELOG on Feature Branch (BEFORE MERGE)

IMPORTANT: All work for the release, including release drafts, must be committed on the feature branch before merging. Drafts are created here so they are available as inputs to the version bump on master; the bump commit can review and modify them.

a. **Create draft release notes file on feature branch:**

- Location: `ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.X.md` (match target version)
- Format: Use previous release notes as template
- Content: Detailed summary of commits on the feature branch since last tag
- Include: Summary, Added/Changed/Fixed sections, Technical Details, Known Issues, Migration Notes, Files Changed, Commits Included
- Generate commit list on feature branch: `git log --oneline <last-tag>..HEAD`
- **Target Audience:** Technical users, developers, detailed changelog

b. **Create draft release announcement file on feature branch:**

- Location: `ReleaseNotes/RELEASE_ANNOUNCEMENT_1.0.0-alpha.X.md`
- Format: Use previous release announcement as template
- Content: User-focused highlights of major changes and bug fixes
- **Target Audience:** End users, highlights only

c. **Create draft CHANGELOG.md entry on feature branch:**

- Add new entry at top of `CHANGELOG.md` (reverse chronological order)
- Version number with date, link to full release notes, Added/Changed/Fixed bullets (concise)

d. **Commit the drafts on the feature branch:**

```
git add ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.X.md ReleaseNotes/RELEASE_ANNOUNCEMENT_1.0.0-
git commit -m "docs(release): add draft release notes and CHANGELOG for v1.0.0-alpha.X"
```

- Ensures no uncommitted changes when checking out master
- Drafts are merged into master with the rest of the feature work

3. Merge Feature Branch into Master (First Commit on Master)

CRITICAL: This creates the FIRST commit on master. The merge commit and version bump commit are SEPARATE commits. Do NOT use `git commit --amend` or any amend commands.

a. **Ensure feature branch is fully committed:**

```
git status  # Verify no uncommitted changes (including release drafts)
```

b. **Checkout master and pull latest:**

```
git checkout master
git pull origin master
```

c. **Merge feature branch into master:**

```
git merge --no-ff feature/branch-name -m "Merge branch 'feature/branch-name' - <short descri
```

- Use `--no-ff` to create a merge commit
- **DO NOT amend this commit** - it must remain separate

d. **Push merged master (FIRST COMMIT):**

```
git push origin master
```

- This pushes the merge commit to master
- All feature branch changes (including release drafts) are now on master

Note: All subsequent steps (version references, review/modify drafts, version bump, tagging) happen on master AFTER the merge. The version bump will be a SECOND commit on master.

4. Update Version References and Finalize Release Materials (On Master)

IMPORTANT: All version references must be updated BEFORE creating the tag. This is a mandatory step. The merged draft release notes, announcement, and CHANGELOG are now on master; review and modify them if needed (e.g. add any commits made directly on master since last tag) before the version bump commit.

a. **Review and, if needed, modify the merged drafts on master:**

- Verify commit list on master: `git log --oneline <last-tag>..HEAD` (includes feature branch + any master-only commits)
- If there were hotfixes or bugfixes committed directly on master since last tag, update the release notes and announcement to include them
- Otherwise the drafts from the feature branch can be used as-is

b. **Update version references** (in this order):

- **Update `contest_tools/version.py`:** `__version__ = "1.0.0-alpha.3"` (match target tag, without 'v' prefix)
- **Update `README.md`:** Line 3: `**Version: 1.0.0-alpha.3**` (MANDATORY for every release; verify no other version references)
- **Update Documentation Files (MANDATORY):**

Reference: See `Docs/VersionManagement.md` Section 3 for complete documentation versioning policy.

Category-Based Update Process:

Category A (User-Facing Documentation):

- Files: `UsersGuide.md`, `InstallationGuide.md`, `ReportInterpretationGuide.md`

- Action: Update version to match project version (e.g., `1.0.0-alpha.3`)
- Update "Last Updated" date to today
- Add revision history entry: "Updated version to match project release v1.0.0-alpha.3 (no content changes)"

Category B (Programmer Reference):

- Files: `ProgrammersGuide.md`, `PerformanceProfilingGuide.md`
- Action: Update version to match project version (e.g., `1.0.0-alpha.3`)
- Update "Last Updated" date to today
- Add revision history entry: "Updated version to match project release v1.0.0-alpha.3 (no content changes)"

Category C (Algorithm Specifications):

- Files: `CallsignLookupAlgorithm.md`, `RunS&PAlgorithm.md`, `WPXPrefixLookup.md`
- Action: Update "Compatible with" field to "up to v1.0.0-alpha.3" (keep algorithm version unchanged)
- Update "Last Updated" date to today
- Add revision history entry: "Updated 'Compatible with' field to include project version v1.0.0-alpha.3"

Category D (Style Guides):

- Files: `CLAReportsStyleGuide.md`
- Action: No version update required (only update when style rules change)
- If style rules changed: Increment version, update "Last Updated" date, add revision history entry

Category E (In-App / Web-Embedded Help):

- Artifacts: `help_dashboard.html`, `help_reports.html`, `help_release_notes.html`, `about.html` (see `Docs/VersionManagement.md` Section 3 Category E)
- Action: **Thorough check** before the version bump commit. Verify: (1) Every help entry point has up-to-date content; (2) Each help template describes current UI (e.g. Rate Differential, Report List, Rate Chart); (3) No stale or removed feature references; (4) In-app links and anchors work. Update templates as needed. Optional: add "Last updated" or version in help footer/About if adopted.
- **When:** During release prep on master (this step); also update on the feature branch when adding or changing UI that affects help.

Verification:

- Verify all Category A & B files match project version
- Verify all Category C files have "Compatible with" field updated
- Verify Category E thorough check completed and in-app help templates updated if needed
- Verify all metadata formats are consistent (see `Docs/VersionManagement.md` Section 3)
- Verify all revision history entries are added (Categories A, B, C)

For complete documentation versioning rules, see: `Docs/VersionManagement.md` Section 3

Note: Release notes, announcement, and CHANGELOG drafts are created on the feature branch (Step 2) and merged in Step 3. Step 4a above covers reviewing and modifying them on master after the merge (e.g. to add any master-only commits). The version bump commit (Step 5) includes these files along with version.py, README, and Docs.

5. Commit Version Updates (Second Commit on Master - MANDATORY BEFORE TAGGING)

CRITICAL: This creates the SECOND commit on master. This commit contains ALL version updates, release notes, documentation versioning, and CHANGELOG.md updates. Do NOT use `git commit --amend` or any amend commands.

IMPORTANT: All version updates, release notes, documentation versioning, and CHANGELOG.md changes MUST be committed as a SINGLE commit BEFORE creating the tag. This commit comes AFTER the merge commit (Step 3) and includes:

- Version number updates (`version.py`, `README.md`)
- Release notes and announcement (drafts from Step 2, possibly reviewed/updated in Step 4)
- Documentation versioning (Category A, B, C updates)
- CHANGELOG.md entry

a. **Stage all version-related changes:**

```
git add contest_tools/version.py README.md ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.3.md Relea
git add Docs/UsersGuide.md Docs/InstallationGuide.md Docs/ReportInterpretationGuide.md
git add Docs/ProgrammersGuide.md Docs/PerformanceProfilingGuide.md
git add Docs/CallsignLookupAlgorithm.md Docs/RunS&PAlgorithm.md Docs/WPXPrefixLookup.md
```

- **Core version files (required):**
  1. `contest_tools/version.py` - Version number
  2. `README.md` - Version in header
  3. `ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.3.md` - Detailed release notes
  4. `ReleaseNotes/RELEASE_ANNOUNCEMENT_1.0.0-alpha.3.md` - User-focused announcement
  5. `CHANGELOG.md` - Changelog entry
- **Documentation files (Category A & B - match project version):** 5. `Docs/UsersGuide.md` 6. `Docs/InstallationGuide.md` 7. `Docs/ReportInterpretationGuide.md` 8. `Docs/ProgrammersGuide.md` 9. `Docs/PerformanceProfilingGuide.md`
- **Algorithm specifications (Category C - update Compatible With field):** 10. `Docs/CallsignLookupAlgorithm.md` 11. `Docs/RunS&PAlgorithm.md` 12. `Docs/WPXPrefixLookup.md`

- **Note:** Category D (Style Guides) only updated when style rules change

b. **Verify all files are staged:**

```
git status
```

- Confirm all required version/release/docs files appear in "Changes to be committed"

c. **Create commit (SECOND COMMIT on master):**

```
git commit -m "chore(release): bump version to 1.0.0-alpha.3 and add release notes"
```

- Use `chore(release):` type for version bumps
- Include "bump version" in message
- Mention release notes and CHANGELOG.md
- **DO NOT use `git commit --amend`** - this must be a separate commit
- This is the commit that will be tagged
- **Note:** Commit message can say "release notes" (plural) to cover both release notes and announcement

d. **Push version bump commit (SECOND COMMIT):**

```
git push origin master
```

- This pushes the version bump commit to master
- Master now has: merge commit → version bump commit

6. Create and Push Tag

a. **Create annotated tag:**

```
git tag -a v1.0.0-alpha.3 -m "Release v1.0.0-alpha.3"
```

- Use `-a` for annotated tag (recommended)
- Tag message can be simple since detailed release notes are available
- Tag name must match version.py (with 'v' prefix)
- **Note:** Detailed release notes are in `ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.3.md`, so tag message can be concise

b. **Verify tag:**

```
git tag -l "v1.0.0-alpha.3"
git show v1.0.0-alpha.3
```

c. **Push tag (user executes):**

```
git push origin v1.0.0-alpha.3  # Push tag
```

- **Note:** The version bump commit should already be pushed (Step 5d)
- Tag points to the version bump commit (second commit on master)

7. Post-Release Verification

a. **Verify version consistency:**

```
# Check version.py matches tag
grep "__version__" contest_tools/version.py
git describe --tags

# Check README matches
grep "Version:" README.md
```

b. **Verify all required files exist and are correct:**

```
# Verify version.py matches tag
grep "__version__" contest_tools/version.py

# Verify README.md updated
grep "Version:" README.md

# Verify release notes exist
ls ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.3.md

# Verify CHANGELOG.md entry exists
grep -A 5 "## \[1.0.0-alpha.3\]" CHANGELOG.md

# Verify version consistency across all files
grep -r "1\.0\.0-alpha\.3" contest_tools/version.py README.md ReleaseNotes/RELEASE_NOTES_1.0

# Verify documentation versions (Category A & B should match project version)
grep "Version:" Docs/UsersGuide.md Docs/InstallationGuide.md Docs/ReportInterpretationGuide.
grep "Version:" Docs/ProgrammersGuide.md Docs/PerformanceProfilingGuide.md

# Verify Category C "Compatible with" fields updated
grep "Compatible with" Docs/CallsignLookupAlgorithm.md Docs/RunS&PAlgorithm.md Docs/WPXPrefi
```

8. Provide User Summary of VPS Update Steps

After the tag is pushed (or as part of release materials in Step 4/5), provide the user with a summary of steps needed to update the VPS server version to match the release.

a. **Create VPS update instructions file:** `ReleaseNotes/VPS_UPDATE_INSTRUCTIONS_1.0.0-alpha.X.md` (match target version). Use a previous release's file as template (e.g. `VPS_UPDATE_INSTRUCTIONS_1.0.0-alpha.14.md`).

b. **Include the four or five script commands to be run on the VPS server:**

1. `cd /docker/Contest-Log-Analyzer` (or user's installation path)
2. `git fetch --tags origin`
3. `git checkout v1.0.0-alpha.X` (the new tag)
4. `docker compose up --build -d`

5. (Optional) Verify: `docker compose ps` and/or `docker compose logs web`

Add a note to adjust path and compose filename if needed. Include brief "What these commands do," "Verify the update," and "If something goes wrong" sections per existing VPS instruction files.

c. **When to create and commit:** Create the file during Step 4 (release prep) and include it in the version bump commit (Step 5), so it is in the repo when the tag is pushed. Alternatively, create and commit it after Step 6 (tag push) as a follow-up commit on master.

d. **Provide to user:** Ensure the user can access the summary (e.g. link in release announcement or release notes to `ReleaseNotes/VPS_UPDATE_INSTRUCTIONS_1.0.0-alpha.X.md`).

AI Agent Responsibilities

- **AI MUST Do (Mandatory Steps):**
  - **Perform document consistency check** (Step 0) - REQUIRED before any updates
  - **Create draft release notes, announcement, and CHANGELOG on feature branch** (Step 2) - Commit them on the feature branch before merge; they are inputs to the version bump
  - **Merge feature branch into master** (Step 3) - Creates FIRST commit on master, push immediately
  - **Review/modify merged drafts on master** (Step 4a) - Add any master-only commits to release notes if needed
  - **Update `contest_tools/version.py`** to match target version (Step 4)
  - **Update `README.md` version** (Line 3) - REQUIRED for every release (Step 4)
  - **Update documentation files** per category-based policy (Step 4) - REQUIRED for every release, included in version bump commit
    - Category A & B: Update version to match project version
    - Category C: Update "Compatible with" field
    - Category D: Only update if style rules changed
    - Category E: Perform thorough check of in-app help (see Version-Management.md Section 3); update templates as needed
  - **Stage all version-related files** (Step 5a): `version.py`, `README.md`, documentation files, `ReleaseNotes/RELEASE_NOTES_*.md`, `ReleaseNotes/RELEASE_ANNOUNCEMENT_*.md`, `CHANGELOG.md`
  - **Commit ALL version updates as SECOND commit** (Step 5c) - Single commit includes version numbers, release notes (possibly updated), documentation versioning, and CHANGELOG.md, push immediately
  - **NEVER use `git commit --amend`** - Always create new commits

  - Generate tag creation command (after version bump commit is pushed)
  - Verify version consistency across all files
  - **Provide user summary of VPS update steps** (Step 8) - Create `ReleaseNotes/VPS_UPDATE_INSTRUCTIONS_1.0.0-alpha.X.md` with the four or five script commands for the VPS; include in version bump commit or commit after tag; provide user with summary (link or list commands)

- **AI Should Do:**
  - Suggest version number based on commit history
  - Generate draft release notes from feature-branch commit history (Step 2)
  - Organize release notes by category (Added, Changed, Fixed)
  - Verify all required files exist before proceeding

- **User Must:**
  - Confirm feature branch is ready for merge
  - Confirm target version number
  - Review release notes content before committing
  - Execute tag push command
  - Verify final state

Critical Reminder: Release notes, announcement, and CHANGELOG drafts are created on the feature branch (Step 2) and committed before merge. They must be included in the version bump commit (Step 5) before tagging the release.

Release Announcement Guidelines:

- **Purpose:** User-focused highlights, less technical than release notes
- **Content:** Major new features, important bug fixes, key improvements
- **Tone:** Accessible, user-friendly language
- **Length:** Shorter than release notes (highlights only)
- **Structure:** Highlights, New Features, Important Fixes, Improvements, link to full release notes
- **Do NOT include:** Technical details, file changes, commit history, implementation details

Version Number Guidelines

- **Alpha versions:** `1.0.0-alpha.X` (e.g., `1.0.0-alpha.3`)
- **Beta versions:** `1.0.0-beta.X` (e.g., `1.0.0-beta.5`)
- **Release candidates:** `1.0.0-rc.X` (e.g., `1.0.0-rc.1`)
- **Releases:** `1.0.0`, `1.1.0`, `2.0.0` (SemVer)

Quick Reference Commands Regular Release:

```
# Step 1: Pre-release checklist
git status  # Verify feature branch is committed (except release drafts)
git describe --tags --abbrev=0  # Find last tag

# Step 2: Create draft release notes, announcement, CHANGELOG on feature branch; commit
# (Edit: ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.X.md, RELEASE_ANNOUNCEMENT_*.md, CHANGELOG.
git add ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.X.md ReleaseNotes/RELEASE_ANNOUNCEMENT_1.0.0-
git commit -m "docs(release): add draft release notes and CHANGELOG for v1.0.0-alpha.X"
git status  # Verify no uncommitted changes

# Step 3: Merge feature branch into master (FIRST COMMIT)
git checkout master
git pull origin master
git merge --no-ff feature/branch-name -m "Merge branch 'feature/branch-name' - <short descri
git push origin master  # Push merge commit

# Step 4: On master - review/modify drafts if needed; update version.py, README, Docs; Categ
# git log --oneline <last-tag>..HEAD  # Verify commits; update release notes if master had
# (Edit: version.py, README.md, Docs Category A/B/C, release notes/CHANGELOG if needed; Cate

# Step 5: Commit ALL version updates (SECOND COMMIT - single commit)
git add contest_tools/version.py README.md ReleaseNotes/RELEASE_NOTES_1.0.0-alpha.X.md Relea
git add Docs/UsersGuide.md Docs/InstallationGuide.md Docs/ReportInterpretationGuide.md
git add Docs/ProgrammersGuide.md Docs/PerformanceProfilingGuide.md
git add Docs/CallsignLookupAlgorithm.md Docs/RunS&PAlgorithm.md Docs/WPXPrefixLookup.md
git commit -m "chore(release): bump version to 1.0.0-alpha.X and add release notes"
git push origin master  # Push version bump commit

# Step 6: Create and push tag (points to version bump commit)
git tag -a v1.0.0-alpha.X -m "Release v1.0.0-alpha.X"
git push origin v1.0.0-alpha.X

# Step 7: Verify version consistency
grep -r "1\.0\.0-alpha\.X" contest_tools/version.py README.md ReleaseNotes/ CHANGELOG.md

# Step 8: Provide user summary of VPS update steps
# Create ReleaseNotes/VPS_UPDATE_INSTRUCTIONS_1.0.0-alpha.X.md (cd, git fetch --tags, git ch
# Include in version bump commit (Step 5) or commit after tag; link in release announcement
```

Hotfix Release:

```
# Create hotfix branch from release tag
git checkout v1.0.0
git checkout -b hotfix/1.0.1-description

# After fix and version bump:
git tag -a v1.0.1 -m "Hotfix v1.0.1"
```

```
git push origin v1.0.1

# Backport to master
git checkout master
git pull origin master
git merge hotfix/1.0.1-description
git push origin master
```

Legacy: Simple Tag Creation (Deprecated)

For quick tags without full release process, see "Before Creating a Tag" section below. However, **full release workflow is recommended** for all version tags.

Before Creating a Tag (Legacy/Quick Method)

1. **User specifies** target version (e.g., `v1.0.0-beta.5`)
2. **AI suggests** updating `version.py`:
   `__version__ = "1.0.0-beta.5"`
3. **User confirms** version number
4. **AI generates** commit message:
   `chore: bump version to 1.0.0-beta.5`
5. **User commits** version change
6. **User creates** tag: `git tag v1.0.0-beta.5`
7. **User pushes** tag: `git push origin v1.0.0-beta.5`

Note: This method does NOT update README.md or create release notes. Use full Release Workflow for proper releases.

Hotfix Workflow: Emergency Releases

Purpose: Handle critical bugs or security vulnerabilities in released versions when master has moved ahead with new features.

When to Use: Critical bug in released version (e.g., `v1.0.0`) but master is at higher version (e.g., `v1.1.0-alpha.1`).

Step-by-Step Hotfix Process 1. Pre-Hotfix Checklist

- ☐ Identify affected release tag (e.g., `v1.0.0`)
- ☐ Confirm bug severity (critical/security/data corruption)
- ☐ Determine patch version (e.g., `v1.0.0` → `v1.0.1`)

2. Create Hotfix Branch from Release Tag

a. **Checkout release tag:**

```
git checkout v1.0.0
```

b. **Create hotfix branch:**

```
git checkout -b hotfix/1.0.1-security-patch
```

3. Fix the Bug

a. **Make minimal fix:**

- Focus only on fixing the bug
- Avoid adding new features
- Test fix thoroughly

b. **Commit fix:**

```
git commit -m "fix(security): resolve authentication bypass"
```

4. Update Version References

IMPORTANT: All version references must be updated BEFORE creating the tag.

a. **Update `contest_tools/version.py`:**

```
__version__ = "1.0.1"  # Patch increment from fixed release
```

b. **Update `README.md` (MANDATORY):**

- Line 3: `**Version: 1.0.1**`
- **MANDATORY:** This must be updated for every release, including hotfixes

5. Create Release Notes and Update CHANGELOG.md (MANDATORY)

IMPORTANT: Release notes and CHANGELOG.md updates are REQUIRED for hotfix releases. These must be created BEFORE creating the tag.

a. **Create release notes file (REQUIRED):**

- Location: `ReleaseNotes/RELEASE_NOTES_1.0.1.md`
- Format: Same as regular release notes
- Content: Focus on bug fix, impact, migration if needed
- Mark as "Hotfix" or "Security" in title
- Include: Summary, Fixed section, Technical Details, Migration Notes

b. **Update CHANGELOG.md (REQUIRED):**

- **MANDATORY:** Add entry at top with `[HOTFIX]` or `[SECURITY]` prefix
- Must include version number with date and link to full release notes
- Example:

```
## [1.0.1] - 2026-01-20 [HOTFIX]
[Full Release Notes](ReleaseNotes/RELEASE_NOTES_1.0.1.md)

### Fixed
- Critical security vulnerability in authentication
```

6. Commit Version Updates (MANDATORY BEFORE TAGGING)

IMPORTANT: All version updates, release notes, and CHANGELOG.md changes MUST be committed BEFORE creating the tag.

a. **Stage all version-related changes:**

```
git add contest_tools/version.py README.md ReleaseNotes/RELEASE_NOTES_1.0.1.md CHANGELOG.md
```

- **All four files must be included:**
  1. `contest_tools/version.py` - Version number
  2. `README.md` - Version in header
  3. `ReleaseNotes/RELEASE_NOTES_1.0.1.md` - Release notes
  4. `CHANGELOG.md` - Changelog entry

b. **Verify all files are staged:**

```
git status
```

c. **Create commit:**

b. **Create commit:**

```
git commit -m "chore(release): bump version to 1.0.1 for hotfix"
```

7. Tag and Push Hotfix

a. **Create annotated tag:**

```
git tag -a v1.0.1 -m "Hotfix v1.0.1

Critical security patch for authentication bypass.

[Summary of fix]
"
```

b. **Push hotfix tag (user executes):**

```
git push origin v1.0.1
```

8. Backport to Master

a. **Checkout master:**

```
git checkout master
git pull origin master
```

b. **Merge hotfix branch:**

```
git merge hotfix/1.0.1-security-patch
```

c. **Resolve conflicts** (if master has diverged significantly)

d. **Push merged master (user executes):**

```
git push origin master
```

Note: Master version remains independent (e.g., `v1.1.0-alpha.1`). The fix is backported and will be included in master's next release.

Version Numbering for Hotfixes

- **Patch increment from fixed release:** `1.0.0` → `1.0.1`
- **For pre-releases:** `1.0.0-alpha.3` → `1.0.0-alpha.4`
- **Master version:** Independent of hotfix version (reflects master's development state)

AI Agent Responsibilities for Hotfixes

- **AI Can Do:**
  - Generate hotfix workflow commands
  - Update `version.py` and `README.md` for hotfix version
  - Generate release notes and CHANGELOG.md entry
  - Create commit message for version bump
  - Generate tag creation command
  - Generate merge commands for backport
- **User Must:**
  - Confirm hotfix is necessary
  - Confirm target version number
  - Review release notes content
  - Execute tag push command
  - Execute master merge push
  - Verify final state

Rollback Strategy: Reverting Bad Releases

Purpose: Handle situations where a release tag needs to be reverted or patched.

When to Rollback

- Release tag contains critical bug
- Security vulnerability discovered after release
- Data corruption issue
- Production outage

Rollback Options Option 1: Create Patch Release (Recommended)

1. Follow Hotfix Workflow
2. Create patch version with fix (e.g., `v1.0.0` → `v1.0.1`)
3. Deploy patch immediately

Option 2: Revert Tag and Create New Release

1. Create revert commit on master:

```
git revert <bad-release-commit>
```

2. Follow Regular Release Workflow
3. Create new release (e.g., `v1.0.1`)

Option 3: Delete and Recreate Tag (Not Recommended)

- Only if tag hasn't been deployed yet
- Warning: Can cause confusion if tag was already pulled

Rollback Process 1. Identify Bad Release

- Tag: `v1.0.0` (contains bug)

2. Determine Strategy

- If fix is ready: Use Hotfix Workflow (create `v1.0.1`)
- If fix needs development: Revert and create new release

3. Execute Rollback

- Follow Hotfix Workflow for immediate fix
- Or revert commit and create new release

4. Update Documentation

- Update release notes to document rollback
- Update CHANGELOG.md with rollback entry
- Document reason for rollback

5. Notify Stakeholders

- Document rollback in release notes
- Include deployment instructions
- Include migration notes if needed

Untagged Commits

- Version string will be: `"1.0.0-beta.5-3-gabc1234"` (most recent tag + distance + hash)
- Git hash always included: `"1.0.0-beta.5 (commit abc1234)"`
- This works for bug reports - version + hash = exact code state

---

Workflow Examples

Example 1: Feature Development

```
User: "Commit these changes with a message about adding the dashboard"

AI:
1. Generates commit message following strict Angular format:
```

feat(dashboard): add user dashboard with widgets

Implements main dashboard view with widget framework. Adds real-time data updates and user preferences.

- Created dashboard view component
- Implemented widget system
- Added API integration for live data

```
2. Executes: git commit with properly formatted message (using multiple -m flags or single -
3. Confirms: "Committed to feat/dashboard branch"
```

Example 2: Merge to Main

```
User: "Merge feat/dashboard into main"

AI:
1. Generates: git checkout main && git pull
2. Generates: git merge --no-ff feat/dashboard
3. Opens editor with merge message:
   "feat(dashboard): add user dashboard with widgets

   Implements main dashboard view with widget framework."
4. Waits for user to review/edit and save
5. Confirms: "Merge complete. Ready to push?"
6. User: "Push it"
7. AI suggests: git push origin main
8. User executes manually
```

Example 3: Creating a Release Tag

```
User: "Tag the next beta release"

AI:
1. Checks recent commits for version bump
2. Suggests: "Based on recent commits, suggest v1.0.0-beta.5"
3. User: "Yes"
4. AI updates version.py __version__ = "1.0.0-beta.5"
5. AI generates: git commit -m "chore: bump version to 1.0.0-beta.5"
6. AI suggests commands:
   git tag v1.0.0-beta.5
   git push origin v1.0.0-beta.5
7. User reviews and executes
```

Error Handling

If Pre-commit Hook Fails

- AI should explain the error
- Do NOT skip hooks without user permission
- Fix the issue (e.g., update version.py manually) before proceeding

If Merge Conflicts

- AI should detect conflicts
- AI should suggest resolution steps
- **User must resolve** conflicts manually
- AI can help review conflict resolution

If Version Mismatch

- AI should warn if `version.py` doesn't match recent tag
- AI should suggest updating version before tagging
- User must confirm version number

If Document Discrepancy Detected

- **CRITICAL:** AI MUST STOP immediately if discrepancy found between `AI_AGENT_RULES.md` and `VersionManagement.md`
- **DO NOT proceed** with version updates
- Report discrepancy using the format in "Version Management" > "CRITICAL: Document Consistency Check" section
- **Wait for explicit user guidance** before proceeding
- **For complete procedures, see:** `Docs/VersionManagement.md` Section 5

Logging Rules

CRITICAL: Log Level Requirements for Docker Visibility

AI agents MUST use WARNING or ERROR level for all logging that needs to appear in Docker logs.

Rule: The project's default logging level is `WARNING` (see `web_app/config/settings.py`):

- `INFO` level logs are **NOT visible** in Docker logs at default settings
- `WARNING` and `ERROR` level logs **ARE visible** in Docker logs at default settings
- `DEBUG` level logs are **NOT visible** in Docker logs at default settings
- Messages at `INFO` or `DEBUG` level will be silently ignored, making debugging impossible

When to Use Each Level:

- `logger.error()`: Critical errors, exceptions, failures that require immediate attention
- `logger.warning()`: Warnings, non-critical issues, diagnostic information that should be visible
- `logger.info()`: Only use when `CLA_PROFILE=1` is explicitly enabled (performance profiling)
- `logger.debug()`: Never use (not visible in Docker logs at default settings)

Agent Behavior:

- **For diagnostic/troubleshooting messages**: Always use `logger.warning()` or `logger.error()`
- **For error conditions**: Use `logger.error()` or `logger.warning()` depending on severity
- **For informational messages**: Only use `logger.info()` if `CLA_PROFILE=1` is enabled
- **Never use `print()` for messages that need to appear in Docker logs** - use logger instead

CRITICAL: Diagnostic Logging Rules

AI agents MUST use WARNING or ERROR level for diagnostic logging, NOT INFO level. AI agents MUST use the `[DIAG]` prefix for ALL diagnostic messages.

Rule: When adding diagnostic logging (for debugging, troubleshooting, or tracing execution flow), AI agents must:

- [**REQUIRED**] **Use `logger.warning()` or `logger.error()`** for all diagnostic messages
- [**REQUIRED**] **Use `[DIAG]` prefix** for all diagnostic messages (mandatory, not optional)
- [**X**] **Never use `logger.info()`** for diagnostics (unless CLA_PROFILE=1 is explicitly enabled)
- [**X**] **Never omit the `[DIAG]` prefix** from diagnostic messages
- [**X**] **Never use `print()`** for diagnostic messages that need to appear in Docker logs

Why This Matters The project's default logging level is `WARNING` (see `web_app/config/settings.py`):

- `INFO` level logs are **NOT visible** at default settings
- `WARNING` and `ERROR` level logs **ARE visible** at default settings
- Diagnostic messages at `INFO` level will be silently ignored, making debugging impossible
- The `[DIAG]` prefix allows easy filtering and identification of diagnostic messages in logs

- `print()` statements go to stdout/stderr but may not be properly formatted or filtered in Docker logs

When Adding Diagnostics Correct Pattern (REQUIRED):

```
# DIAGNOSTICS: Log request details (ERROR level for visibility, [DIAG] prefix required)
logger.error(f"[DIAG] analyze_logs called. Method: {request.method}, Path: {request.path}")
logger.error(f"[DIAG] POST request detected. FILES keys: {list(request.FILES.keys())}")
logger.warning(f"[DIAG] Form validation FAILED. Form errors: {form.errors}")
```

Incorrect Patterns (DO NOT USE):

```
# WRONG: INFO level logs won't be visible at default settings
logger.info(f"Manual upload detected. FILES keys: {list(request.FILES.keys())}")
logger.info(f"Form validation failed: {form.errors}")

# WRONG: Missing [DIAG] prefix (required for all diagnostic messages)
logger.warning(f"Processing {call} {mult_name} for sum_by_band")
logger.error(f"Form validation failed: {form.errors}")

# WRONG: print() statements won't be properly formatted in Docker logs
print(f"DEBUG: Form submission diagnostics")
print(f"  CONTENT_TYPE={content_type}")

# CORRECT: Use logger.warning() instead
logger.warning(f"[DIAG] Form submission diagnostics")
logger.warning(f"  CONTENT_TYPE={content_type}")
```

Best Practices

1. **[REQUIRED] Use `[DIAG]` prefix** for ALL diagnostic messages - this is mandatory, not optional
2. **Use `logger.error()` for critical diagnostics** (entry points, error paths, validation failures)
3. **Use `logger.warning()` for non-critical diagnostics** (state transitions, optional information)
4. **Always assume default log level is WARNING** - never assume `INFO` will be visible
5. **Add diagnostics at entry points** of functions to trace execution flow
6. **Wrap potentially failing operations** in try/except with error-level logging
7. **Format: `[DIAG] <descriptive message>`** - always start with `[DIAG]` followed by a space

Example: Comprehensive Diagnostic Logging

```
def analyze_logs(request):
    # DIAGNOSTICS: Log all requests (ERROR level for visibility)
```

```
logger.error(f"[DIAG] analyze_logs called. Method: {request.method}, Path: {request.path

if request.method == 'POST':
    logger.error(f"[DIAG] POST request detected. FILES keys: {list(request.FILES.keys())

    # Wrap potentially failing operations
    try:
        _update_progress(request_id, 1)
        logger.error(f"[DIAG] _update_progress called successfully")
    except Exception as e:
        logger.error(f"[DIAG] _update_progress FAILED: {e}")
        logger.exception("Exception in _update_progress")

    # Add diagnostics at key decision points
    if 'log1' in request.FILES:
        logger.error(f"[DIAG] Manual upload branch entered")
        # ... rest of code ...
    elif 'fetch_callsigns' in request.POST:
        logger.error(f"[DIAG] Public fetch branch entered")
        # ... rest of code ...
    else:
        logger.error(f"[DIAG] Neither branch taken. Redirecting to home.")
```

Agent Behavior When adding diagnostic logging:

1. [**REQUIRED**] **Always** use `logger.warning()` or `logger.error()` for diagnostics
2. [**REQUIRED**] **Always** prefix diagnostic messages with `[DIAG]` (mandatory, not optional)
3. [**X**] **Never** use `logger.info()` for diagnostics (unless explicitly required)
4. [**X**] **Never** omit the `[DIAG]` prefix from diagnostic messages
5. **Add** diagnostics at function entry points and key decision points
6. **Wrap** potentially failing operations in try/except with error logging
7. **Assume** default log level is WARNING - ensure diagnostics will be visible

Enforcement: AI agents must include `[DIAG]` in every diagnostic log message. This prefix is required for:

- Easy filtering of diagnostic messages in logs (e.g., `grep "[DIAG]" logs.txt`)
- Distinguishing diagnostics from actual warnings/errors
- Consistent logging format across the codebase
- Enabling automated log analysis tools to identify diagnostic output

Rationale

- **Visibility:** WARNING/ERROR level ensures diagnostics are always vis-

ible, regardless of log level configuration
- **Debugging:** Entry-point and error-path logging enables effective troubleshooting
- **Consistency:** Standardized diagnostic pattern makes logs easier to read and filter
- **Reliability:** Assuming WARNING-level visibility prevents silent diagnostic failures

CRITICAL: Do Not Remove Diagnostic Logging During Debugging

AI agents MUST NOT remove diagnostic logging until a bug fix is tested and verified to work OR explicitly directed to remove it.

Rule: When debugging issues, AI agents must:

- **[REQUIRED] Keep diagnostic logging in place** during the debugging process
- **[REQUIRED] Wait for user confirmation** that the bug fix works before removing diagnostics
- **[REQUIRED] Only remove diagnostics** when:
  1. The bug fix has been tested and verified to work correctly, OR
  2. The user explicitly directs removal of diagnostic logging
- **[X] Never remove diagnostic logging** as part of the initial bug fix implementation
- **[X] Never remove diagnostic logging** without user confirmation that the fix works

Why This Matters:

- Diagnostic logging is essential for verifying that bug fixes actually work
- Removing diagnostics too early can make it impossible to verify the fix
- User needs to see diagnostic output to confirm the issue is resolved
- Diagnostic logging helps identify if the fix introduced new issues

Workflow:

1. **Add diagnostic logging** to identify the bug
2. **Implement bug fix** while keeping diagnostics
3. **Test and verify** the fix works (diagnostics help confirm)
4. **Wait for user confirmation** that the fix is verified
5. **Only then remove diagnostics** (or keep them if user requests)

Exception: If user explicitly requests removal of diagnostic logging at any point, AI agents may remove it as directed.

Agent Behavior:

- When fixing bugs, keep all diagnostic logging added during debugging
- Do not remove diagnostic logging as part of the fix implementation
- Wait for user to test and verify the fix before removing diagnostics

- Only remove diagnostics after explicit user confirmation or direction

---

Debugging Rules

CRITICAL: No Inferring Bug Sources

AI agents MUST NOT infer or guess the root cause of bugs. Add temporary diagnostic output to confirm the actual failing path or value, then fix based on that evidence.

MANDATORY WORKFLOW:

1. **Reproduce** the bug (or note the user report).
2. **Add temporary diagnostics** to confirm where/when/why it fails:
   - Use `logger.warning()` or `logger.error()` so output is visible (e.g. in Docker logs).
   - Use the `[DIAG]` prefix for all diagnostic messages (see "Logging Rules" > "CRITICAL: Diagnostic Logging Rules" above).
   - Log the actual values (e.g. keys, paths, IDs) at the call site where the bug manifests and at the place that provides the disputed data.
3. **Use the evidence** from logs/output to drive the fix. Do not fix based on reasoning alone without confirming.
4. **Remove or gate diagnostics** when done (e.g. remove, comment out, or guard with a DEBUG flag). Do not leave temporary diagnostics in place unless the user requests otherwise.

Rule: When debugging, do not infer or guess. Add targeted diagnostics (warning/error level, [DIAG] prefix), confirm the actual cause, then fix. Remove or gate diagnostics when the fix is complete.

UI / Dashboard bugs: When the symptom is "nothing shows" or "wrong data in UI" (e.g. missing plots, wrong selector), consider **both** backend (data, keys, paths, context) and frontend (selectors, data attributes, default state, JS keys). Add diagnostics in both layers if the cause is unclear; confirm the mismatch with logs before changing code.

See also: `.cursor/rules/debugging.mdc` for a concise reminder.

Safety Checks

Before any critical operation, AI should:

1. [OK] Confirm current branch (`git branch`)
2. [OK] Show recent commits (`git log --oneline -5`)
3. [OK] Warn if on main branch for direct commits
4. [OK] Suggest creating feature branch if not on one

5. [OK] Verify pre-commit hooks are installed

CRITICAL: Stash Before Cleaning/Reverting Changes

IMPORTANT: Uncommitted changes exist in the working directory, NOT in branches. When you switch branches, uncommitted changes travel with you. If you discard changes on one branch (e.g., `git restore .` or `git clean -fd`), those changes are PERMANENTLY LOST from ALL branches.

Rule: Before performing any destructive git operations that might discard uncommitted changes, AI agents MUST:

1. **Check for uncommitted changes:**

   ```
   git status
   ```

2. **If uncommitted changes exist, STASH them first:**

   ```
   git stash push -m "WIP: [description of changes]"
   ```

3. **Perform the operation** (e.g., clean master branch, switch branches, etc.)

4. **Restore changes on target branch:**

   ```
   git checkout [target-branch]
   git stash pop
   ```

Common Scenarios Requiring Stash:

- **Cleaning master/main branch while on feature branch:**
  - **WRONG:** `git checkout master && git restore . && git clean -fd`
  - **CORRECT:** `git stash push -m "WIP: feature changes" && git checkout master && git restore . && git clean -fd && git checkout feature-branch && git stash pop`
- **Switching branches with uncommitted changes:**
  - If changes should stay on current branch: Stash first, switch, then pop on return
  - If changes should move to new branch: Just switch (git carries them), but be aware they're not committed
- **Reverting changes on one branch while keeping them on another:**
  - Stash changes first
  - Switch to branch to clean
  - Clean that branch
  - Switch back to feature branch
  - Pop stash to restore changes

Why This Matters:

- Uncommitted changes are NOT stored in branches - they're in the working directory
- Discarding changes on one branch discards them everywhere
- Stashing preserves changes in git's stash, allowing safe branch operations
- Stash can be restored on any branch with `git stash pop` or `git stash apply`

Example Workflow: Moving Changes from Master to Feature Branch:

```
# 1. Check status (on master with uncommitted changes)
git status

# 2. Stash the changes
git stash push -m "WIP: ARRL DX implementation"

# 3. Verify master is clean
git status  # Should show "working tree clean"

# 4. Create and switch to feature branch
git checkout -b feature/1.0.0-alpha.10

# 5. Restore stashed changes
git stash pop

# 6. Verify changes are restored
git status  # Should show the uncommitted changes again
```

Rationale:

- Prevents accidental loss of uncommitted work
- Allows safe branch operations without losing changes
- Makes it clear where changes belong (which branch)
- Enables clean separation between branches

Work Tracking System

Three-Track System

AI agents MUST understand the three-track work tracking system:

1. **Technical Debt** (`TECHNICAL_DEBT.md`): Issues with existing code that need fixing
2. **Future Work** (`DevNotes/FUTURE_WORK.md`): Committed work we will do (decided to proceed)
3. **Potential Enhancements** (`DevNotes/POTENTIAL_ENHANCEMENTS.md`): Ideas we might do (not committed)

CRITICAL: Category Identification Requirement

AI agents MUST identify which category an item belongs to when:

- User proposes a new feature or enhancement
- User identifies an issue or problem
- User discusses potential improvements
- AI agent identifies something that needs tracking

If category is unclear, AI agent MUST ask the user to clarify:

- "Is this a bug/issue with existing code? (Technical Debt)"
- "Is this a committed enhancement we will do? (Future Work)"
- "Is this an idea we might do? (Potential Enhancements)"

AI agents MUST NOT add items to tracking documents without identifying the correct category.

Technical Debt Tracking

AI agents MUST check `TECHNICAL_DEBT.md` at the start of each session.

When TECHNICAL_DEBT.md Exists:

- AI agent **must** read and review technical debt items at session start
- AI agent **should** prioritize work that addresses technical debt
- AI agent **should** update `TECHNICAL_DEBT.md` as items are completed
- AI agent **should** add new items when technical debt is identified

File Location: Repository root (`TECHNICAL_DEBT.md`)

Purpose: Track technical debt items, architectural improvements, and known issues that need attention.

What Belongs Here:

- Bugs in existing code
- Architectural problems
- Inconsistencies in current implementation
- Performance issues with existing features
- Code quality problems

Categories:

- Architecture & Design Debt
- Validation & Testing Debt
- Code Quality Debt
- Documentation Debt
- Performance Debt
- Known Issues

Agent Behavior:

1. **Session Start:** Read `TECHNICAL_DEBT.md` (non-blocking but recommended)
2. **During Work:** Check off items as completed, add new items as discovered
3. **Priority:** Address HIGH priority items when relevant to current task
4. **Category Identification:** When adding items, ensure they are bugs/issues with existing code, not new features

---

Future Work Tracking

AI agents SHOULD check `DevNotes/FUTURE_WORK.md` when planning work.

When FUTURE_WORK.md Exists:

- AI agent **should** review future work items when planning implementations
- AI agent **should** update status as work progresses
- AI agent **should** link to implementation plans when created

File Location: `DevNotes/FUTURE_WORK.md`

Purpose: Track committed work that we have decided to proceed with.

What Belongs Here:

- Committed enhancements we will implement
- Work that has been decided to proceed with
- Items moved from Potential Enhancements after decision to proceed

Status Values:

- Planned - Committed to doing, not yet started
- In Progress - Currently being implemented
- Completed - Implemented and verified
- Deferred - Postponed (with reason)
- Cancelled - Decided not to proceed (with reason)

Agent Behavior:

1. **When Planning:** Review future work items relevant to current task
2. **When Implementing:** Update status to "In Progress"
3. **When Completing:** Update status to "Completed" and link to implementation
4. **Category Identification:** Only add items here if they are committed work (decided to proceed)

Potential Enhancements Tracking

AI agents SHOULD check `DevNotes/POTENTIAL_ENHANCEMENTS.md` when discussing new features.

When POTENTIAL_ENHANCEMENTS.md Exists:

- AI agent **should** review potential enhancements when discussing new features
- AI agent **should** add items when new capabilities are proposed
- AI agent **should** update status as items are evaluated

File Location: `DevNotes/POTENTIAL_ENHANCEMENTS.md`

Purpose: Track ideas, proposals, and potential enhancements that are under consideration but not yet committed.

What Belongs Here:

- Ideas and proposals
- Potential new capabilities
- Enhancements under consideration
- Research items

Status Values:

- Proposed - Initial idea, needs discussion
- Under Discussion - Being evaluated/designed
- Deferred - Postponed for later consideration (with reason)
- Rejected - Decided not to implement (with reason)

Agent Behavior:

1. **When Discussing:** Add items here when new ideas are proposed
2. **When Evaluating:** Update status as items are discussed
3. **When Deciding:** Move to Future Work if decided to proceed, or mark Deferred/Rejected
4. **Category Identification:** Add items here if they are ideas/proposals, not committed work

---

Session Context & Workflow State

HANDOVER.md Pattern (Optional)

For complex multi-session workflows, a `HANDOVER.md` file may exist at the repository root to maintain workflow state between agent sessions.

When HANDOVER.md Exists:

- AI agent **must** check for `HANDOVER.md` at session start
- AI agent **must** read and incorporate workflow state from `HANDOVER.md`

- AI agent **should** update `HANDOVER.md` as work progresses
- AI agent **should** overwrite `HANDOVER.md` when switching sessions

When HANDOVER.md Does NOT Exist:

- AI agent proceeds with standard context (git history, documentation, code, user request)
- AI agent does NOT create `HANDOVER.md` unless explicitly requested
- Standard workflow: User request → Agent analyzes code/docs → Agent performs task

When to Use HANDOVER.md:

- [OK] Complex multi-session workflows (e.g., version cleanup, major refactoring)
- [OK] Multi-step processes spanning multiple sessions (test → patch → tag → cleanup)
- [OK] Context that spans multiple sessions (current state, next steps, decisions)

When NOT to Use HANDOVER.md:

- ☒ Simple bug fixes (agent can analyze code directly)
- ☒ Single-session tasks (new features, quick fixes)
- ☒ Tasks with sufficient context in code/docs/git history

Agent Behavior:

1. **Session Start:** Check for `HANDOVER.md` (non-blocking)
2. **If Found:** Read workflow state, incorporate into context
3. **If Not Found:** Proceed with standard context gathering
4. **During Work:** Update `HANDOVER.md` if it exists and workflow state changes
5. **Session End:** Overwrite `HANDOVER.md` with current state if it exists

File Location: Repository root (`HANDOVER.md`)

Git Treatment: Committed but overwritten (allows backup in git history while maintaining current state)

File Organization Rules

Test and Temporary Scripts Location

AI agents MUST place all test, diagnostic, and temporary scripts in the `test_code/` directory or a subdirectory thereof.

Rule: Scripts that are not part of the production codebase must be located in `test_code/`:

- **Correct:** `test_code/diagnose_score_discrepancy.py`, `test_code/utils/helper_script.py`

- **Incorrect:** `contest_tools/utils/diagnose_score_discrepancy.py`, `tools/debug_script.py` (outside `test_code/`)

Applies to:

- Diagnostic scripts (e.g., `diagnose_score_discrepancy.py`, `trace_multiplier_logic.py`)
- Temporary debugging scripts
- One-off analysis scripts
- Test utilities and helpers
- Batch files for running diagnostics (e.g., `run_all_diagnostics.bat`)
- Mockup generators and mockup output files (e.g., `test_code/mockups/generate_enhanced_missed_mul`

Does NOT apply to:

- Production utilities in `contest_tools/utils/` (e.g., `architecture_validator.py`, `report_utils.py`)
- Production tools in `tools/` (e.g., `update_version_hash.py`, `setup_git_aliases.ps1`)
- Test suites in `tests/` (if using pytest/unittest structure)
- `__main__` blocks in core utilities (e.g., `contest_tools/core_annotations/get_cty.py`) - These are debugging/testing features, not standalone scripts

Rationale:

- Keeps production code directories clean
- Separates temporary/diagnostic code from production utilities
- Makes it clear which files are part of the shipped codebase
- Aligns with existing project structure (`test_code/` already contains similar scripts)

Agent Behavior:

- When creating diagnostic/temporary scripts, place them in `test_code/` or `test_code/subdirectory/`
- **Standard location for temporary scripts:** `regression_baselines/test_code/temp_scripts/` (gitignored, auto-generated)
- When creating mockup generators or mockup files, place them in `test_code/mockups/`
- Do NOT add such scripts to `contest_tools/utils/`, `tools/`, or other production directories
- If a script needs to become production code, refactor and move it appropriately (e.g., `contest_tools/utils/`)

CRITICAL: Main Project Must Not Reference Test Infrastructure

Rule: The main project code (`contest_tools/`, `web_app/`) must NEVER import from or depend on test infrastructure (`test_code/`).

- **Correct:** Test utilities in `test_code/` import from main project (`contest_tools/`, `web_app/`)
- **Incorrect:** Main project code importing from `test_code/`

- **Rationale:** Main project should be independent and testable. Test infrastructure is a consumer of the main project, not a dependency.

Examples:

- `test_code/web_regression_test.py` imports from `contest_tools.log_manager`
- `test_code/regression_test_utils.py` is in `test_code/` (not `contest_tools/utils/`)
- `contest_tools/utils/regression_test_utils.py` (would create dependency from main project to test code)
- `web_app/analyzer/views.py` importing from `test_code/`

Discussion Documents Location

Purpose: Standardize location for architectural discussions, design decisions, and planning documents.

Rule: All discussion documents, architectural analysis documents, implementation plans, and design decision documents **MUST** be placed in the `DevNotes/` directory.

Examples of Documents That Belong in DevNotes:

- Architectural discussion documents (e.g., `COMPONENT_ARCHITECTURE_DISCUSSION.md`)
- Refactoring analysis documents (e.g., `DASHBOARD_REFACTORING_DISCUSSION.md`)
- Implementation planning documents (e.g., `PHASE1_SEQUENCING_DECISION.md`)
- Design decision records
- Technical feasibility studies
- Project planning documents

Examples of Documents That Do NOT Belong in DevNotes:

- User-facing documentation (goes in `Docs/`)
- Release notes (goes in `ReleaseNotes/` or `RELEASE_NOTES_*.md` at root)
- Code documentation (goes with code files or `Docs/`)
- README files (typically at directory roots where they belong)

Agent Behavior:

- When creating new discussion/planning documents, place them in `DevNotes/`
- When moving existing discussion documents, move them to `DevNotes/`
- Use descriptive filenames (e.g., `COMPONENT_ARCHITECTURE_DISCUSSION.md`, not `discussion.md`)

Directory Structure:

```
DevNotes/
    LAQP_Implementation_Plan.md
    DASHBOARD_REFACTORING_DISCUSSION.md
```

```
PHASE1_SEQUENCING_DECISION.md
COMPONENT_ARCHITECTURE_DISCUSSION.md
... (other discussion/planning documents)
```

Code Architecture Rules

CRITICAL: Data Aggregation Layer (DAL) Pattern

AI agents MUST follow the DAL pattern when implementing or modifying reports.

Core Principle: Reports are visualization/formatting layers. Data processing belongs exclusively in aggregators.

Rules for Reports (`contest_tools/reports/`)

1. **DO NOT process raw log data in reports.** Use aggregators from `contest_tools.data_aggregators` instead.
2. **DO NOT create pivot tables, groupby operations, or manual time binning in reports.** All data aggregation belongs in aggregators.
3. **DO NOT manually iterate over logs to extract or transform data.** Use aggregator methods that handle all logs consistently.
4. **DO use `master_time_index` from `LogManager`** when passing time_index parameters to aggregators to ensure alignment across logs.
5. **DO use existing aggregator methods** (e.g., `MatrixAggregator.get_stacked_matrix_data()`, `TimeSeriesAggregator.get_time_series_data()`). If new data structures are needed, extend aggregators with new methods rather than processing data in reports.

Why This Matters

- **Time Alignment:** Manual time binning in reports causes misalignment across logs, leading to rendering bugs where only some logs appear correctly.
- **Consistency:** Aggregators ensure all logs are processed identically, preventing data structure mismatches.
- **Maintainability:** Centralized data processing logic is easier to test, debug, and extend.
- **Performance:** Aggregators can be cached and reused across multiple reports.

Example: Correct Pattern

```python
# CORRECT: Report uses aggregator
def generate(self, output_path: str, **kwargs):
    # Get master_time_index for alignment
    master_index = None
    if self.logs and hasattr(self.logs[0], '_log_manager_ref'):
        log_manager = self.logs[0]._log_manager_ref
        if log_manager:
            master_index = log_manager.master_time_index

    # Use aggregator - do NOT process raw data
    matrix_agg = MatrixAggregator(self.logs)
    matrix_data = matrix_agg.get_stacked_matrix_data(bin_size='60min', time_index=master_inc

    # Report only formats/visualizes the pre-processed data
    # ... visualization code using matrix_data ...
```

Example: Anti-Pattern (DO NOT DO THIS)

```python
# WRONG: Report manually processes data
def generate(self, output_path: str, **kwargs):
    for log in self.logs:
        df = get_valid_dataframe(log)
        # BAD: Manual pivot tables, time binning, etc. in report
        pivot = df.pivot_table(index='Band', columns='Hour', ...)
        # This causes time misalignment and inconsistent data structures!
```

Reference Documentation For detailed information on the DAL pattern and available aggregators, see:

- `Docs/ProgrammersGuide.md` - Section 2: "The Data Aggregation Layer (DAL)"
- Individual aggregator modules in `contest_tools/data_aggregators/`

CRITICAL: Contest-Specific Plugin Architecture

AI agents MUST respect contest-specific plugins and calculators as the single source of truth.

Core Principle: "Contest-specific algorithmic weirdness is implemented in code (plugins). Common declarative rules defined in contest definitions are handled by core modules."

Refined Boundary:

- **Core modules** (like `StandardCalculator`) handle declarative rules from contest definitions:
  - `applies_to` filtering, `totaling_method` variations, standard score formulas
  - These are metadata-driven patterns defined in JSON
- **Plugins** handle algorithmic complexity:
  - Complex stateful calculations, non-standard formulas, custom algorithms
  - True "weirdness" that requires custom implementation

See PLUGIN_ARCHITECTURE_BOUNDARY_DECISION.md for detailed discussion.

Rules for Plugin Architecture

1. **DO NOT bypass contest-specific plugins or calculators.**
   - If a contest defines a `time_series_calculator` (e.g., `wae_calculator`, `naqp_calculator`), use it as the authoritative source for final scores
   - If a contest defines a `scoring_module` (e.g., `arrl_10_scoring`), respect it for point calculation
   - Do NOT create alternative calculation paths that ignore plugins
2. **DO use plugins as single source of truth.**
   - Score calculators (`time_series_calculator`) are authoritative for final scores and cumulative score arrays
   - Scoring modules (`scoring_module`) are authoritative for QSO point calculation
   - Aggregators should use plugin output when available, not recalculate independently
3. **DO validate architectural compliance before proposing solutions.**
   - Before proposing changes, verify: Does this respect plugin architecture?
   - Before creating aggregators, verify: Should this use an existing plugin?
   - Before modifying scoring, verify: Does this respect contest-specific calculators?
4. **DO add validation warnings if multiple calculation paths exist.**
   - If aggregator calculates independently of plugin, add validation check
   - Warn if plugin and aggregator results differ significantly
   - Use plugin as authoritative source, aggregator as validation/fallback only

Architecture Decision Framework When making changes that affect data calculation or contest-specific logic:

Step 1: Identify Architecture Requirements

- Does contest have plugin/calculator defined in JSON?
- What is the authoritative source for this calculation?
- Is this algorithmic complexity (needs plugin) or declarative rule (core module handles)?
- Are there existing patterns to follow?

Step 2: Verify Compliance

- Does my solution use the plugin/calculator (if algorithmic complexity)?
- For declarative rules (applies_to, totaling_method): Should core module handle this?
- Am I creating alternative paths that bypass architecture?
- Are multiple calculation paths necessary (with validation)?

Step 3: Validate Consistency

- If multiple paths exist, add validation checks using `ArchitectureValidator`
- Use plugin as authoritative source
- Warn if results differ significantly (> 1 point tolerance)

Step 4: Document Decision

- Document why this approach respects architecture
- Explain if fallback paths are needed
- Note any validation checks added

Example: Correct Pattern (Respects Plugin)

```
# CORRECT: Using calculator as source of truth
if hasattr(log, 'time_series_score_df') and not log.time_series_score_df.empty:
    # Use plugin output as authoritative
    final_score = int(log.time_series_score_df['score'].iloc[-1])
else:
    # Fallback only if plugin unavailable
    final_score = calculate_fallback_score(log)
```

Example: Anti-Pattern (Bypasses Plugin)

```
# WRONG: Bypassing contest-specific calculator
# ScoreStatsAggregator calculates score independently, ignoring time_series_calculator
final_score = total_points * total_mults  # Doesn't use log.time_series_score_df
# This violates plugin architecture!
```

Example: Validation Pattern (Multiple Paths with Checks)

```
# CORRECT: Multiple paths with validation
from contest_tools.utils.architecture_validator import ArchitectureValidator
```

```python
validator = ArchitectureValidator()
warnings = validator.validate_scoring_consistency(log)
if warnings:
    for warning in warnings:
        logging.warning(f"Architecture validation: {warning}")

# Use plugin as authoritative source
if hasattr(log, 'time_series_score_df'):
    final_score = int(log.time_series_score_df['score'].iloc[-1])
```

Plugin Architecture Checklist Before implementing scoring/data changes:

- ☐ Does contest have `time_series_calculator` or `scoring_module` defined?
- ☐ Is plugin output available (e.g., `log.time_series_score_df`)?
- ☐ Does my solution use plugin as authoritative source?
- ☐ Am I creating an alternative calculation path that bypasses plugins?
- ☐ Should I add validation to detect inconsistencies?

Why This Matters

- **Contest-Specific Logic:** Plugins implement contest-specific scoring rules (e.g., WAE, NAQP, WRTC)
- **Single Source of Truth:** Ensures all reports/dashboards show consistent scores
- **Maintainability:** Contest-specific logic is centralized in plugins, not scattered across aggregators
- **Extensibility:** New contests can add plugins without modifying core aggregators

Reference Documentation

- `DevNotes/SCORING_ARCHITECTURE_ANALYSIS.md` - Detailed analysis of scoring architecture
- `contest_tools/score_calculators/` - Score calculator implementations
- `contest_tools/contest_specific_annotations/` - Scoring module implementations
- `contest_tools/utils/architecture_validator.py` - Architecture validation utility

Python Script Execution Rules

CRITICAL: Use Miniforge Python for Script Execution

AI agents MUST use the miniforge Python executable when running Python scripts in this environment.

MANDATORY: Check this section BEFORE executing any Python command. Do NOT assume `python` will work.

MANDATORY WORKFLOW:

1. **BEFORE executing any Python command:** Check this section of AI Agent Rules
2. **ALWAYS use the full path:** `C:\Users\mbdev\miniforge3\python.exe`
3. **NEVER use:** `python`, `python.exe`, or `py` without the full path
4. **AFTER execution:** Check exit code and error messages (see "Command Execution and Error Checking" section)

Rule: When executing Python scripts, always use the full path to the miniforge Python executable:

- **Correct:** `C:\Users\mbdev\miniforge3\python.exe script.py`
- **Incorrect:** `python script.py` or `python.exe script.py` (these do not work in this environment)

Canonical Commands (Single Source of Truth) Use these exact invocations. Do not invent alternatives. See also `.cursor/rules/project-mandatory.mdc`.

Action	Command
Run a Python script	`C:\Users\mbdev\miniforge3\python.exe script_pat`
Run pytest	`C:\Users\mbdev\miniforge3\python.exe -m pytest`
Run py_compile (after modifying any Python file)	`C:\Users\mbdev\miniforge3\envs\cla\python.exe -`
Django management command	`C:\Users\mbdev\miniforge3\python.exe web_app/ma`

Before running Python/tests checklist:

- ☐ Read this "Python Script Execution Rules" section
- ☐ Use full path: `C:\Users\mbdev\miniforge3\python.exe` for scripts and pytest
- ☐ Use full path: `C:\Users\mbdev\miniforge3\envs\cla\python.exe` for py_compile
- ☐ Never use `python`, `python.exe`, or `py` without the full path

Why This Matters The default `python` command does not resolve correctly in this environment. Using the miniforge Python path ensures:

- Scripts execute with the correct Python interpreter (Python 3.12.10 from conda-forge)
- All dependencies and modules are properly accessible
- Consistent execution environment across all script runs

Examples Correct Python Script Execution:

```
# Running a script from test_code directory
C:\Users\mbdev\miniforge3\python.exe test_code/verify_engine.py

# Running a script with arguments
C:\Users\mbdev\miniforge3\python.exe -m pytest tests/

# Running Python one-liners
C:\Users\mbdev\miniforge3\python.exe -c "import sys; print(sys.version)"

# Running Django management commands
C:\Users\mbdev\miniforge3\python.exe web_app/manage.py runserver
```

Incorrect Python Script Execution (DO NOT USE):

```
# These will fail or not work correctly
python script.py
python.exe script.py
py script.py
```

Agent Behavior MANDATORY WORKFLOW FOR ALL PYTHON COMMANDS:

1. **BEFORE execution:**
   - Check this section of AI Agent Rules
   - Identify the required Python path: `C:\Users\mbdev\miniforge3\python.exe`
   - Use this path in the command
2. **DURING execution:**
   - **Always** use `C:\Users\mbdev\miniforge3\python.exe` as the Python interpreter
   - **Never** use `python`, `python.exe`, or `py` without the full path
   - **Preserve** relative paths for script arguments (e.g., `test_code/script.py`)
   - **Use** full path only for the Python executable itself
3. **AFTER execution:**
   - Check exit code (non-zero = failure)
   - Read error messages completely
   - If error: Stop, fix, retry with correct path
   - If success: Verify output is as expected
   - Do NOT proceed with dependent operations if command failed

Note on Activation Scripts The `C:\Users\mbdev\miniforge3\Scripts\custom_activate.bat` script uses `/K` flag which opens an interactive terminal window. For non-interactive script execution, use the Python executable directly rather than trying to activate the environment first.

CRITICAL: Syntax Validation Before Finishing

AI agents MUST validate Python syntax using `py_compile` before completing code changes.

MANDATORY: Check Python syntax after modifying any Python file.

MANDATORY WORKFLOW:

1. **AFTER modifying any Python file:** Run `py_compile` to check for syntax errors
2. **ALWAYS use the conda environment Python:** `C:\Users\mbdev\miniforge3\envs\cla\python.e -m py_compile <file_path>`
3. **NEVER skip this step:** Syntax errors cause runtime failures and must be caught immediately
4. **FIX all syntax errors** before proceeding with other work

Rule: After creating or modifying any Python file, AI agents must:

- **Run:** `C:\Users\mbdev\miniforge3\envs\cla\python.exe -m py_compile <file_path>`
- **Check exit code:** Non-zero exit code indicates syntax errors
- **Read error messages:** Syntax errors show exact line numbers and issues
- **Fix immediately:** Do not proceed until all syntax errors are resolved

Why This Matters:

- Catches indentation errors, missing colons, unclosed brackets, and other syntax issues
- Prevents runtime failures that would break the application
- Identifies errors immediately rather than during execution
- Saves debugging time by catching issues early

Example Workflow:

```
# After modifying a Python file
C:\Users\mbdev\miniforge3\envs\cla\python.exe -m py_compile regression_baselines\test_code\w

# If successful (exit code 0): Proceed with other work
# If failed (exit code non-zero): Read error message, fix syntax, retry compilation
```

Note: Use the conda environment Python (`envs\cla\python.exe`) to ensure the same Python version and dependencies as the project uses.

Common Syntax Errors to Catch:

- IndentationError (mismatched indentation levels)
- SyntaxError (missing colons, unclosed brackets, invalid syntax)
- TabError (mixing tabs and spaces)
- Missing closing brackets, parentheses, or quotes

Agent Behavior:

- **MUST** run `py_compile` after modifying any Python file
- **MUST** fix all syntax errors before proceeding
- **MUST** verify exit code is zero before considering work complete
- **MUST NOT** skip syntax validation even for "simple" changes

---

PowerShell Command Syntax Rules

CRITICAL: PowerShell vs CMD Syntax Differences

AI agents MUST use PowerShell syntax when executing commands in this environment, NOT CMD syntax.

MOST COMMON ERROR: Using `&&` instead of `;` for command chaining

- **WRONG:** `cd dir && git init` (causes error: "The token '&&' is not a valid statement separator")
- **CORRECT:** `cd dir; git init`

Rule: This environment uses PowerShell (`powershell.exe`), not Windows Command Prompt (`cmd.exe`). AI agents must use PowerShell-compatible syntax for all terminal commands.

Quick Reference - Command Chaining: | Syntax | PowerShell (CORRECT) | CMD (INCORRECT - DO NOT USE) | |--------|---------------------|---------------------------| | Chain commands | `cmd1; cmd2; cmd3` | `cmd1 && cmd2 && cmd3` | | Example | `cd dir; git status` | `cd dir && git status` |

Why This Matters PowerShell and CMD have different syntax for:

- Command chaining (`&&` vs `;`)
- Executing batch files (`call` vs `&` or direct execution)
- Path handling and directory changes
- Variable expansion and quoting

Using CMD syntax in PowerShell causes errors like:

- `The token '&&' is not a valid statement separator in this version`
- `call : The term 'call' is not recognized as the name of a cmdlet, function, script file, or operable program`
- Path resolution errors when changing directories

Command Chaining PowerShell Syntax (CORRECT):

```
# Use semicolon (;) to chain commands
cd regression_baselines; git init; git config user.name "Test User"
```

```
# Or use separate commands on separate lines
cd regression_baselines
git init
git config user.name "Test User"
```

CMD Syntax (INCORRECT - DO NOT USE):

```
# WRONG: && is CMD syntax, not PowerShell
cd regression_baselines && git init && git config user.name "Test User"
```

Note: For file operations (move, copy, delete), see "File Operation Verification" section under "Command Execution and Error Checking" for complete verification requirements.

Executing Batch Files PowerShell Syntax (CORRECT):

```
# Direct execution (if path has no spaces)
regression_baselines\test_setup.bat

# Using call operator (if path has spaces or special characters)
& "regression_baselines\test_setup.bat"

# Using full path with call operator
& "C:\Users\mbdev\OneDrive\Desktop\Repos\Contest-Log-Analyzer\regression_baselines\test_setu
```

CMD Syntax (INCORRECT - DO NOT USE):

```
# WRONG: call is CMD syntax, not PowerShell
call regression_baselines\test_setup.bat
```

Directory Changes PowerShell Syntax (CORRECT):

```
# Use Set-Location or cd with proper path handling
Set-Location regression_baselines
# Or
cd regression_baselines

# Use absolute paths when current directory is uncertain
Set-Location "C:\Users\mbdev\OneDrive\Desktop\Repos\Contest-Log-Analyzer\regression_baseline

# Use -C flag for git commands to specify directory (avoids cd issues)
git -C regression_baselines init
git -C regression_baselines status
```

Common Issues:

- If already in `regression_baselines`, `cd regression_baselines` will fail (looking for `regression_baselines/regression_baselines`)
- Use `git -C <directory>` to avoid directory change issues

- Check current directory with `Get-Location` or `pwd` before changing directories

Path Handling PowerShell Syntax (CORRECT):

```
# Use backslashes for Windows paths (PowerShell accepts both, but backslashes are standard)
$path = "regression_baselines\Logs"

# Use quotes for paths with spaces
$path = "C:\Users\mbdev\OneDrive\Desktop\Repos\Contest-Log-Analyzer\regression_baselines"

# Use Join-Path for path construction
$logPath = Join-Path $scriptDir "Logs"
```

Variable Expansion PowerShell Syntax (CORRECT):

```
# Use $variable for variables
$TEST_DATA_DIR = "regression_baselines\Logs"
echo $TEST_DATA_DIR

# Use $env:VARIABLE for environment variables
echo $env:TEST_DATA_DIR
```

CMD Syntax (INCORRECT - DO NOT USE):

```
# WRONG: %VARIABLE% is CMD syntax
set TEST_DATA_DIR=regression_baselines\Logs
echo %TEST_DATA_DIR%
```

Command Execution and Error Checking CRITICAL: AI agents MUST check command outputs and handle errors before proceeding.

CRITICAL: File operations (move, copy, delete, rename) require verification of both source and destination states. See "File Operation Verification" subsection below.

MANDATORY WORKFLOW FOR ALL COMMANDS - NO EXCEPTIONS:

BEFORE execution (MANDATORY):

1. **Check relevant project rules FIRST:** Review AI Agent Rules for command-specific requirements. See also `.cursor/rules/` for mandatory project rules (Python paths, debugging).
   - Python commands: Check "Python Script Execution Rules" section → Use `C:\Users\mbdev\miniforge3\python.exe`
   - **PowerShell commands: CRITICAL - Check "PowerShell Command Syntax Rules" section**

    - **NEVER use `&&` for command chaining** - This is CMD syntax and will FAIL in PowerShell
    - **ALWAYS use `;` for command chaining** - This is PowerShell syntax
    - **Example WRONG:** `cd dir && git init` (will cause error: "The token '&&' is not a valid statement separator")
    - **Example CORRECT:** `cd dir; git init`
    - See "PowerShell Command Syntax Rules" section (line ~1809) for complete details
  - Git commands: Check git workflow rules
2. **Use correct syntax/paths:** Apply rules from project documentation - do NOT assume defaults will work
   - **PowerShell syntax check:** If chaining commands, verify you're using `;` not `&&`
3. **Verify prerequisites:** Ensure required files/directories exist before executing

AFTER execution (MANDATORY):

1. **Check exit code IMMEDIATELY:**
   - Non-zero exit code = FAILURE → STOP, read error, fix, retry
   - Zero exit code = SUCCESS → BUT STILL VERIFY RESULTS (exit code 0 does NOT guarantee success for file operations)
2. **For file operations (move/copy/delete/rename):** ALWAYS verify both source and destination states (see "File Operation Verification" below)
3. **Read error messages COMPLETELY:** Error messages contain critical information - read the entire error, not just the first line
4. **Verify command results:** Check that commands produced expected output (not empty, not unexpected format)
5. **Address failures IMMEDIATELY:**
   - Do NOT proceed with subsequent commands if a command fails
   - Do NOT ignore errors and continue
   - Do NOT assume "it will work next time"
   - Do NOT assume exit code 0 means success for file operations
6. **Check working directory:** Verify current directory when path-related errors occur
7. **Do NOT ignore errors:** Empty output or unexpected results require investigation before proceeding

Rule: After executing any terminal command, AI agents MUST:

1. **Check exit codes:** Non-zero exit codes indicate failure
2. **Read error messages:** Error messages contain critical information about what went wrong
3. **Verify command results:** Check that commands produced expected output

4. **Address failures immediately:** Do not proceed with subsequent commands if a command fails
5. **Check working directory:** Verify current directory when path-related errors occur

Error Checking Workflow:

```
# Example: Check if file exists
$result = Test-Path "test_code\regression_test_utils.py"
if (-not $result) {
    # File doesn't exist - investigate why
    Write-Host "ERROR: File not found. Checking current directory..."
    Get-Location
    # Fix the issue before proceeding
}
```

Common Error Patterns to Watch For:

- **Path not found errors:** `Cannot find path '...' because it does not exist`
  - **Action:** Check current working directory with `Get-Location` or `pwd`
  - **Action:** Use absolute paths or verify relative paths are correct
  - **Action:** Verify the file/directory actually exists before using it
- **Exit code non-zero:** Commands return exit code > 0
  - **Action:** Read the error message in the command output
  - **Action:** Do not proceed with dependent operations
  - **Action:** Fix the underlying issue before continuing
- **Empty output when expecting results:** Commands return no output
  - **Action:** Verify the command syntax is correct
  - **Action:** Check if the operation actually succeeded
  - **Action:** Use verbose flags or additional checks to verify success

Example Error Handling:

```
# BAD: Ignoring errors
Test-Path "test_code\regression_test_utils.py"
Get-ChildItem "test_code"  # This might fail if path is wrong

# GOOD: Checking and handling errors
$fileExists = Test-Path "test_code\regression_test_utils.py"
if (-not $fileExists) {
    Write-Host "ERROR: File not found at test_code\regression_test_utils.py"
    Write-Host "Current directory: $(Get-Location)"
    Write-Host "Checking if test_code directory exists..."
    if (Test-Path "test_code") {
        Get-ChildItem "test_code" | Select-Object Name
```

```
    } else {
        Write-Host "ERROR: test_code directory not found. Wrong working directory?"
    }
    # DO NOT PROCEED until issue is resolved
}
```

Working Directory Verification:

```
# Always verify working directory when path errors occur
$currentDir = Get-Location
Write-Host "Current directory: $currentDir"

# If wrong directory, change to correct one
if ($currentDir -notlike "*Contest-Log-Analyzer*") {
    Set-Location "C:\Users\mbdev\OneDrive\Desktop\Repos\Contest-Log-Analyzer"
}
```

CRITICAL: File Operation Verification

AI agents MUST verify file operations (move, copy, delete, rename) by checking both source and destination states.

MANDATORY: After ANY file operation, verify the operation actually succeeded.

File Operation Verification Checklist:

BEFORE execution:

- ☐ Source file/directory exists (use `Test-Path`)
- ☐ Destination directory exists (use `Test-Path` for directory)
- ☐ Using PowerShell cmdlets (`Move-Item`, `Copy-Item`, `Remove-Item` - NOT `move`, `copy`, `del`)
- ☐ Using PowerShell syntax (`;` for chaining, NOT `&&`)

AFTER execution:

- ☐ Exit code checked (non-zero = stop immediately, even if some files moved)
- ☐ Source state verified: For moves/deletes, file should be GONE from source
- ☐ Destination state verified: For moves/copies, file should EXIST in destination
- ☐ Both verifications passed before proceeding to next operation

MANDATORY WORKFLOW FOR FILE OPERATIONS:

1. **BEFORE operation:**
    - Verify source file/directory exists using `Test-Path`
    - Verify destination directory exists using `Test-Path`
    - Use PowerShell cmdlets (`Move-Item`, `Copy-Item`, `Remove-Item`) - NOT CMD commands (`move`, `copy`, `del`)

- Use PowerShell syntax (`;` for chaining) - NOT CMD syntax (`&&`)

2. **EXECUTE operation:**
   - Use proper PowerShell cmdlet with correct syntax
   - Use `-Force` flag if overwriting is intended
3. **AFTER operation:**
   - **Check exit code:** Non-zero = stop immediately, investigate, fix
   - **Verify source state:**
     - For moves/deletes: File should be GONE from source location
     - For copies: File should still exist in source location
   - **Verify destination state:**
     - For moves/copies: File should EXIST in destination location
     - For deletes: File should NOT exist (verify deletion)
   - **Use Test-Path for verification:** Don't assume - actually check both locations
   - **Only proceed if both verifications pass**

Example: File Move with Verification

```powershell
# BEFORE: Verify prerequisites
$source = "DevNotes\VERSIONING_IMPLEMENTATION_SUMMARY.md"
$dest = "DevNotes\Archive\VERSIONING_IMPLEMENTATION_SUMMARY.md"

# Verify source exists
if (-not (Test-Path $source)) {
    Write-Host "ERROR: Source file does not exist: $source"
    exit 1
}

# Verify destination directory exists
$destDir = Split-Path $dest -Parent
if (-not (Test-Path $destDir)) {
    Write-Host "ERROR: Destination directory does not exist: $destDir"
    exit 1
}

# EXECUTE: Perform move operation
Move-Item $source $dest -Force

# AFTER: Verify operation succeeded
# Check exit code was already done by tool, but verify file states:

# Verify source is GONE (for moves)
if (Test-Path $source) {
    Write-Host "ERROR: Move failed - source file still exists: $source"
    exit 1
```

```
}

# Verify destination EXISTS (for moves)
if (-not (Test-Path $dest)) {
    Write-Host "ERROR: Move failed - destination file does not exist: $dest"
    exit 1
}

# Only proceed if both verifications pass
Write-Host "SUCCESS: File moved and verified"
```

Example: Batch File Operations with Verification

```
# For multiple files, verify each operation
$files = @("file1.md", "file2.md", "file3.md")
$destDir = "DevNotes\Archive"

# Verify destination exists
if (-not (Test-Path $destDir)) {
    Write-Host "ERROR: Destination directory does not exist: $destDir"
    exit 1
}

# Move and verify each file
foreach ($file in $files) {
    $source = "DevNotes\$file"
    $dest = "$destDir\$file"

    # Verify source exists
    if (-not (Test-Path $source)) {
        Write-Host "WARNING: Source file does not exist: $source (skipping)"
        continue
    }

    # Execute move
    Move-Item $source $dest -Force

    # Verify move succeeded
    if (Test-Path $source) {
        Write-Host "ERROR: Move failed - source still exists: $source"
        exit 1
    }
    if (-not (Test-Path $dest)) {
        Write-Host "ERROR: Move failed - destination missing: $dest"
        exit 1
    }
}
```

```
Write-Host "SUCCESS: All files moved and verified"
```

Common Mistakes to Avoid:

- **WRONG:** Assuming exit code 0 means file operation succeeded
- **WRONG:** Not checking if source file was actually removed (for moves/deletes)
- **WRONG:** Not checking if destination file actually exists (for moves/copies)
- **WRONG:** Using CMD commands (`move`, `copy`, `del`) instead of PowerShell cmdlets
- **WRONG:** Using CMD syntax (`&&` for chaining) instead of PowerShell syntax (`;`)
- **WRONG:** Proceeding with subsequent operations after unverified file operations
- **WRONG:** Ignoring error messages even when exit code is 0

Why This Matters:

- File operations can appear to succeed but actually fail silently
- Directory context issues can cause operations to fail in unexpected ways
- Partial success (some files moved, others failed) can cause inconsistent state
- Verification prevents cascading failures from unverified operations
- Exit code 0 does NOT guarantee file operation succeeded - always verify
- Ensures file operations actually completed before proceeding with dependent operations

Agent Behavior Requirements:

When performing file operations:

1. **MUST** verify source exists before operation
2. **MUST** verify destination directory exists before operation
3. **MUST** use PowerShell cmdlets (`Move-Item`, `Copy-Item`, `Remove-Item`) - NOT CMD commands
4. **MUST** use PowerShell syntax (`;` for chaining) - NOT CMD syntax (`&&`)
5. **MUST** check exit code after operation
6. **MUST** verify source state after operation (gone for moves/deletes, unchanged for copies)
7. **MUST** verify destination state after operation (exists for moves/copies, gone for deletes)
8. **MUST** use `Test-Path` for verification - don't assume
9. **MUST NOT** proceed with subsequent operations if verification fails
10. **MUST NOT** ignore error messages even if exit code is 0

Agent Behavior Requirements:

When executing terminal commands in this environment:

1. **Always** use PowerShell syntax (`;` for chaining, `&` for batch files, `$variable` for variables)
2. **Never** use CMD syntax (`&&` for chaining, `call` for batch files, `%variable%` for variables)
3. **Prefer** `git -C <directory>` over `cd <directory>` to avoid path issues
4. **Check** current directory with `Get-Location` or `pwd` before changing directories
5. **Use** absolute paths or `Set-Location` with proper path handling when directory changes are needed
6. **Use** `&` operator or direct execution for batch files, never `call`
7. **MUST check exit codes** and error messages after every command
8. **MUST read error output** completely before proceeding
9. **MUST verify working directory** when path-related errors occur
10. **MUST fix errors** before executing dependent commands
11. **MUST NOT ignore** empty output or unexpected results

Quick Reference

Operation	PowerShell (CORRECT)	CMD (INCORRECT)
Chain commands	`cmd1; cmd2; cmd3`	`cmd1 && cmd2 && cmd3`
Execute batch file	`& "path\file.bat"` or `path\file.bat`	`call path\file.bat`
Set variable	`$var = "value"`	`set var=value`
Use variable	`$var` or `$env:VAR`	`%var%` or `%VAR%`
Change directory	`Set-Location path` or `cd path`	`cd path`
Git in directory	`git -C path command`	`cd path && git command`

Rationale

- **Environment Consistency:** This environment uses PowerShell, so all commands must use PowerShell syntax
- **Error Prevention:** Using CMD syntax causes command failures and requires manual correction
- **User Experience:** Correct syntax ensures commands execute successfully without user intervention
- **Maintainability:** Consistent PowerShell syntax makes scripts and commands easier to understand and maintain

---

Django Template Syntax Rules

CRITICAL: Template Tag Nesting Constraints

AI agents MUST understand Django template tag nesting limitations to avoid syntax errors.

Rule: `{% else %}` Cannot Be Used Between `{% if %}` and `{% for %}`
Problem: Django's template parser does not allow `{% else %}` between `{% if %}` and `{% for %}` tags. This causes a `TemplateSyntaxError`:

```
TemplateSyntaxError: Invalid block tag on line X: 'else', expected 'empty' or 'endfor'
```

Incorrect Pattern (DO NOT USE):

```
{% if condition %}
    {% for item in items %}
{% else %}
    {% for item in other_items %}
{% endif %}
    <!-- loop content -->
{% endfor %}
```

Correct Solutions:

Option 1: Duplicate Loop Code (Recommended for Simple Cases)

```
{% if condition %}
    {% for item in items %}
        <!-- loop content -->
    {% endfor %}
{% else %}
    {% for item in other_items %}
        <!-- loop content -->
    {% endfor %}
{% endif %}
```

Option 2: Use `{% with %}` to Set Variable First

```
{% if condition %}
    {% with data=items %}
{% else %}
    {% with data=other_items %}
{% endif %}
    {% for item in data %}
        <!-- loop content -->
    {% endfor %}
    {% endwith %}
```

Option 3: Prepare Data in View

```
# In view: prepare data based on condition
context['dimension_data'] = low_modes_data + high_modes_data if is_mode_dimension else low_
```

```
{# In template: use prepared data #}
{% for block in dimension_data %}
    <!-- loop content -->
{% endfor %}
```

Why This Matters

- **Syntax Errors:** Invalid template syntax causes runtime errors that break the application
- **User Experience:** Template errors prevent pages from rendering, causing 500 errors
- **Debugging:** Template syntax errors can be confusing because the error message may not clearly indicate the nesting issue

Agent Behavior When working with Django templates:

1. **Avoid** using `{% else %}` between `{% if %}` and `{% for %}` tags
2. **Prefer** duplicating loop code if the loop content is simple
3. **Consider** preparing data in the view if the logic is complex
4. **Test** template syntax by checking for linter errors or running the Django development server

Rule: `{% else %}` Cannot Be Used Inside `{% with %}` Block **Problem:** Django's template parser does not allow `{% else %}` inside a `{% with %}` block. This causes a `TemplateSyntaxError`:

```
TemplateSyntaxError: Invalid block tag on line X: 'else', expected 'endwith'. Did you forget
```

Incorrect Pattern (DO NOT USE):

```
{% with total_unique=stat.unique_run|add:stat.unique_sp|add:stat.unique_unk %}
{% with scale_max=fixed_multiplier_max|default:row.max_unique %}
{% if is_sweepstakes %}
    {% with total_for_bar=stat.count %}
{% else %}
    {% with total_for_bar=total_unique %}
{% endif %}
    <!-- content using total_for_bar -->
{% endwith %}
{% endwith %}
{% endwith %}
```

Correct Solution: Duplicate Code in Each Branch

```
{% with total_unique=stat.unique_run|add:stat.unique_sp|add:stat.unique_unk %}
{% with scale_max=fixed_multiplier_max|default:row.max_unique %}
{% if is_sweepstakes %}
    {% with total_for_bar=stat.count %}
        <!-- content using total_for_bar -->
    {% endwith %}
{% else %}
    {% with total_for_bar=total_unique %}
        <!-- content using total_for_bar -->
    {% endwith %}
```

```
{% endif %}
{% endwith %}
{% endwith %}
```

Key Points:

- Each branch (`{% if %}` and `{% else %}`) must have its own complete `{% with %}` block
- Each `{% with %}` block must be closed with `{% endwith %}` before the `{% else %}`
- Outer `{% with %}` blocks can remain open for both branches

Rule: Parentheses Not Supported in `{% if %}` Statements Problem: Django's template parser does not support parentheses for grouping conditions in `{% if %}` statements. This causes a `TemplateSyntaxError`:

```
TemplateSyntaxError: Could not parse the remainder: '(row.label' from '(row.label'
```

Incorrect Pattern (DO NOT USE):

```
{% if not (row.label == "TOTAL" and multiplier_count == 1) %}
    <!-- content -->
{% endif %}
```

Correct Solution: Use De Morgan's Law to Rewrite Without Parentheses

Rewrite the condition using logical equivalences:

- `not (A and B)` is equivalent to `(not A) or (not B)`
- `not (A or B)` is equivalent to `(not A) and (not B)`

Example 1: Negated AND

```
{# Incorrect: {% if not (row.label == "TOTAL" and multiplier_count == 1) %} #}
{# Correct: #}
{% if row.label != "TOTAL" or multiplier_count != 1 %}
    <!-- content -->
{% endif %}
```

Example 2: Negated OR

```
{# Incorrect: {% if not (condition1 or condition2) %} #}
{# Correct: #}
{% if not condition1 and not condition2 %}
    <!-- content -->
{% endif %}
```

Example 3: Complex Conditions

```
{# Incorrect: {% if (A and B) or (C and D) %} #}
{# Correct: Use nested if statements or prepare in view #}
```

```
{% if A and B %}
    <!-- content -->
{% elif C and D %}
    <!-- content -->
{% endif %}
```

Key Points:

- Django templates do NOT support parentheses for grouping in `{% if %}` statements
- Use De Morgan's law to rewrite negated conditions
- For complex conditions, use nested `{% if %}` statements or prepare the condition in the view
- When in doubt, simplify the condition or move the logic to Python

Related Issues

- Similar constraints may apply to other template tag combinations
- When in doubt, duplicate code or move logic to the view
- Django template syntax is strict - follow Django documentation patterns

---

DevNotes Directory Structure

AI agents MUST understand the DevNotes organization:

Directories AI Agents Should Read

- **Root DevNotes/**: Active planning documents - READ these for current work
- **Decisions_Architecture/**: Important patterns, architectural decisions, and reusable guidance - READ these

Directories AI Agents Should Ignore (Unless Requested)

- **Archive/**: Completed implementation plans - IGNORE unless user explicitly requests historical context
- **Discussions/**: Obsolete/complete discussions - IGNORE unless user explicitly requests historical context

When to Review Archive/Discussions

- User explicitly asks: "How was X implemented?" or "Why did we choose Y?"
- AI agent needs historical context to understand a decision
- Researching implementation details of completed work
- User requests review of specific historical discussion

File Status Standardization

All DevNotes files should have standardized metadata:

```
**Status:** Active | Completed | Deferred | Obsolete
**Date:** YYYY-MM-DD
**Last Updated:** YYYY-MM-DD
**Category:** Planning | Discussion | Decision | Pattern
```

Value Test for Categorization

When categorizing DevNotes files:

- **Decisions_Architecture/**: Documents reusable patterns, architectural principles, or important decisions that future work should follow
- **Discussions/**: Historical conversations where decisions were made (the decision itself may be documented elsewhere)
- **Archive/**: Completed step-by-step implementation plans (historical reference)
- **Root DevNotes/**: Active planning documents for current or deferred work

Directory Structure

```
DevNotes/
  [Active Planning Documents]          # Active work - AI agents read these
  Archive/                              # Completed plans - AI agents ignore
  Discussions/                          # Obsolete/complete discussions - AI agents ignore
  Decisions_Architecture/               # Important decisions/patterns - AI agents should
```

AI Agent Operation Checklists

CRITICAL: Checklist Maintenance Rule

MANDATORY: When `AI_AGENT_RULES.md` is modified, AI agents MUST:

1. **Review the modified sections** in `AI_AGENT_RULES.md`
2. **Check `Docs/AI_AGENT_CHECKLISTS.md`** for affected checklists
3. **Update relevant checklists** to match new or changed rules
4. **Verify checklist items align** with mandatory workflows in the source document
5. **Update source line references** if document structure changed
6. **Test checklist completeness** against recent operations

Purpose: The checklists in `Docs/AI_AGENT_CHECKLISTS.md` are quick-reference tools derived from this document. They must remain synchronized with the source rules to ensure AI agents follow correct workflows. Key

checklists include: Python Script Execution (Checklist 4), Debugging / Bug Investigation (Checklist 5), Terminal Command Execution (Checklist 3).

Location: `Docs/AI_AGENT_CHECKLISTS.md`

When to Update:

- Any time `AI_AGENT_RULES.md` is modified
- When new mandatory workflows are added
- When existing workflows are changed
- When section line numbers change (update references)

Verification: After updating checklists, verify they cover all mandatory steps from the source document.

Summary

AI's Role: Generate, suggest, assist, format **User's Role:** Review, approve, execute, control **Balance:** AI efficiency + Human oversight = Safe, consistent workflow